STAR RX72N - rx_frame_ascii Library

1.0.0

Generated by Doxygen 1.16.1

Chapter 1

Topic Index

1.1 Topics

Here is a list of all topics with brief descriptions:

Chapter 2

Directory Hierarchy

2.1 Directories

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

Chapter 4

Topic Documentation

4.1 Internal Helper Functions

String building helper functions with bounds checking.

Functions

- RX_STATIC_TESTABLE uint32_t internal_append_str (char *buf, uint32_t pos, uint32_t max, const char *str)

  Append string to buffer with bounds checking.
- RX_STATIC_TESTABLE uint32_t internal_append_char (char *buf, uint32_t pos, uint32_t max, char c)

  Append single character to buffer with bounds checking.
- RX_STATIC_TESTABLE uint32_t internal_append_hex_byte (char *buf, uint32_t pos, uint32_t max, uint8_t byte)

  Append byte as two hex characters to buffer.
- RX_STATIC_TESTABLE uint32_t internal_append_hex_u16 (char *buf, uint32_t pos, uint32_t max, uint16_t value)

  Append uint16_t as four hex characters to buffer.
- RX_STATIC_TESTABLE uint32_t internal_append_hex_u32 (char *buf, uint32_t pos, uint32_t max, uint32_t value)

  Append uint32_t as eight hex characters to buffer.
- RX_STATIC_TESTABLE uint32_t internal_append_decimal_u16 (char *buf, uint32_t pos, uint32_t max, uint16_t value)

  Append uint16_t as decimal digits to buffer.
- RX_STATIC_TESTABLE bool internal_is_printable (uint8_t c)

  Check if byte value is printable ASCII character.
- static uint32_t internal_format_hex_line (char *buf, uint32_t pos, uint32_t max, const uint8_t *payload, uint16_t offset, uint16_t line_bytes)

  Format payload as hex dump with ASCII sidebar.
- RX_STATIC_TESTABLE uint32_t internal_format_header (char *buf, uint32_t pos, uint32_t max, const rx_frame_header_t *header, bool is_tx)

  Format frame header metadata as single text line.
- RX_STATIC_TESTABLE uint32_t internal_append_flag (char *buffer, uint32_t pos, uint32_t max, bool *first, uint8_t flags, uint8_t flag_bit, const char *flag_name)

  Append a single flag name with pipe separator if needed.

4.1.1 Detailed Description

String building helper functions with bounds checking.

All append helpers follow a consistent signature pattern for composability:
uint32_t internal_append_xxx(char* buf, uint32_t pos, uint32_t max, <value>);

Parameters

- buf: Output buffer pointer (may be NULL for safety)
- pos: Current write position in buffer
- max: Maximum buffer size (total capacity)
- value: Data to append (varies by function)

Return Value

All helpers return the updated position after appending. If the append would overflow, the original position is returned unchanged.

Bounds Safety

- All functions check pos < max - k_null_terminator before writing
- NULL buffer pointer causes immediate return with unchanged position
- Zero-size buffer causes immediate return with unchanged position
- No writes beyond max - 1 to preserve space for null terminator

Usage Pattern

```
uint32_t pos = 0;
pos = internal_append_str(buf, pos, max, "Hello ");
pos = internal_append_decimal_u16(buf, pos, max, 42);
pos = internal_append_char(buf, pos, max, '!');
buf[pos] = '\0';  // Null terminate
// Result: "Hello 42!"
```

4.1.2 Function Documentation

4.1.2.1 internal_append_char()

```
RX_STATIC_TESTABLE uint32_t internal_append_char (
        char * buf,
        uint32_t pos,
        uint32_t max,
        char c)
```

Append single character to buffer with bounds checking.

Writes a single character to the buffer at the current position if space is available (accounting for null terminator).

Precondition

pos < max - 1 (for writing to occur)

Postcondition

buf[pos] contains c (if written)

Since

Version 1.0.0

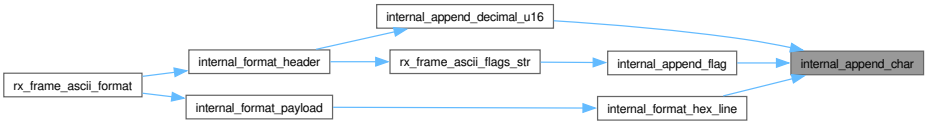
Definition at line 335 of file rx_frame_ascii.c.

```
00336 {
00337   /* Rule 5: Pre-condition invariant - callers guarantee these conditions */
00338   RX_ASSERT(buf != nullptr && max != k_empty_buffer && pos < max,
00339             "internal_append_char: precondition violated by caller");
00340
00341   if (pos < max - k_null_terminator) {
00342     buf[pos++] = c;
00343   }
00344   return pos;
00345 }
```

References k_empty_buffer, and k_null_terminator.

Referenced by internal_append_decimal_u16(), internal_append_flag(), and internal_format_hex_line().

Here is the caller graph for this function:



4.1.2.2 internal_append_decimal_u16()

```
RX_STATIC_TESTABLE uint32_t internal_append_decimal_u16 (
                char * buf,
                uint32_t pos,
                uint32_t max,
                uint16_t value)
```

Append uint16_t as decimal digits to buffer.

Converts a 16-bit value to its decimal string representation (1-5 digits). Uses a small stack buffer for digit reversal, avoiding heap allocation. Zero value outputs single '0' character.

Algorithm

1. Extract digits via repeated modulo/division (reverse order)
2. Store in temporary stack buffer
3. Reverse and append to output buffer

Precondition

Sufficient space for up to 5 digits

Postcondition

Decimal digits written without leading zeros (except for value=0)

Example Output

- value=0 -> "0"
- value=1 -> "1"
- value=12345 -> "12345"
- value=65535 -> "65535"

Stack Usage

~6 bytes (temp buffer for digit reversal)

Since

Version 1.0.0

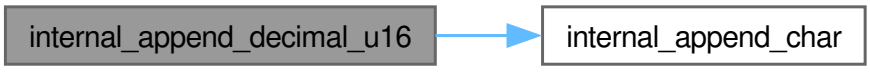
Definition at line 488 of file rx_frame_ascii.c.

```
00492 {
00493   /* Rule 5: Pre-condition invariant - callers guarantee these conditions */
00494   RX_ASSERT(buf != nullptr && max != k_empty_buffer && pos < max,
00495             "internal_append_decimal_u16: precondition violated by caller");
00496
00497   char    temp[k_u16_decimal_buf_size]; /* Max 5 digits + null for uint16_t */
00498   uint8_t len = 0;
00499
00500   if (value == 0) {
00501     return internal_append_char(buf, pos, max, '0');
00502   }
00503
00504   while (value > 0) {
00505     const uint8_t digit = (uint8_t)(value % k_decimal_base);
00506     temp[len++]         = (char)('0' + digit);
00507     value /= k_decimal_base;
00508   }
00509
00510   /* Reverse digits into output */
00511   while (len > 0) {
00512     pos = internal_append_char(buf, pos, max, temp[--len]);
00513   }
00514
00515   return pos;
00516 }
```

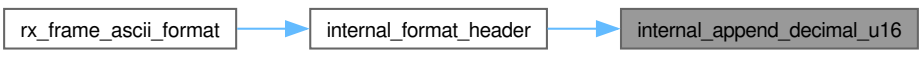
References internal_append_char(), k_decimal_base, k_empty_buffer, and k_u16_decimal_buf_size.

Referenced by internal_format_header().

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.3 internal_append_flag()

```
RX_STATIC_TESTABLE uint32_t internal_append_flag (
                char * buffer,
                uint32_t pos,
                uint32_t max,
                bool * first,
                uint8_t flags,
                uint8_t flag_bit,
                const char * flag_name)
```

Append a single flag name with pipe separator if needed.

Conditionally appends a flag name to the output buffer if the corresponding flag bit is set. Manages separator insertion: no separator before first flag, pipe separator before subsequent flags.

Example Sequence

1. flags=ACK|PRI, first call (first=true): appends "ACK", sets first=false
2. Same flags, second call: appends "|PRI"

Precondition

first != nullptr

Postcondition

*first set to false if flag was appended

Separator '|' prepended if not first flag

Since

Version 1.0.0

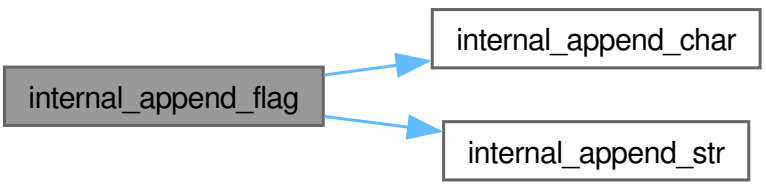
Definition at line 806 of file rx_frame_ascii.c.

```
00813 {
00814   /* Rule 5: Pre-condition invariant - callers always pass valid local bool address */
00815   RX_ASSERT(first != nullptr, "internal_append_flag: first is nullptr - caller must validate");
00816
00817   if (flags & flag_bit) {
00818     if (!*first) {
00819       pos = internal_append_char(buffer, pos, max, '|');
00820     }
00821     pos    = internal_append_str(buffer, pos, max, flag_name);
00822     *first = false;
00823   }
00824   return pos;
00825 }
```

References internal_append_char(), and internal_append_str().

Referenced by rx_frame_ascii_flags_str().

Here is the call graph for this function:



Here is the caller graph for this function:

4.1.2.4 internal_append_hex_byte()

```
RX_STATIC_TESTABLE uint32_t internal_append_hex_byte (
                char * buf,
                uint32_t pos,
                uint32_t max,
                uint8_t byte)
```

Append byte as two hex characters to buffer.

Converts a single byte to its two-character hexadecimal representation (e.g., 0xFF -> "FF") and appends it to the buffer. Uses uppercase hex letters via s_hex_chars lookup table.

Precondition

pos + 2 <= max - 1 (for writing to occur)

Postcondition

buf[pos] contains upper nibble hex char

buf[pos+1] contains lower nibble hex char

Example Output

- byte=0x00 -> "00"
- byte=0xFF -> "FF"
- byte=0x5A -> "5A"

Since

Version 1.0.0

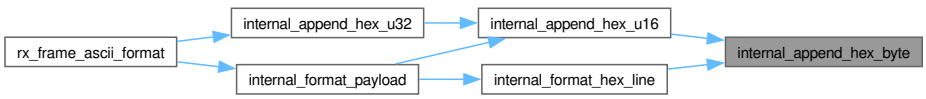
Definition at line 370 of file rx_frame_ascii.c.

```
00374 {
00375   /* Rule 5: Pre-condition invariant - callers guarantee these conditions */
00376   RX_ASSERT(buf != nullptr && max >= k_hex_chars_per_byte && pos < max,
00377             "internal_append_hex_byte: precondition violated by caller");
00378
00379   if (pos < max - k_hex_chars_per_byte) {
00380     buf[pos++] = s_hex_chars[(byte >> k_hex_digit_shift) & k_hex_digit_mask];
00381     buf[pos++] = s_hex_chars[byte & k_hex_digit_mask];
00382   }
00383   return pos;
00384 }
```

References k_hex_chars_per_byte, k_hex_digit_mask, k_hex_digit_shift, and s_hex_chars.

Referenced by internal_append_hex_u16(), and internal_format_hex_line().

Here is the caller graph for this function:



4.1.2.5 internal_append_hex_u16()

```
RX_STATIC_TESTABLE uint32_t internal_append_hex_u16 (
                char * buf,
                uint32_t pos,
                uint32_t max,
                uint16_t value)
```

Append uint16_t as four hex characters to buffer.

Converts a 16-bit value to its four-character hexadecimal representation (e.g., 0xABCD -> "ABCD") in big-endian order (most significant byte first).

Precondition

pos + 4 <= max - 1 (for writing to occur)

Postcondition

Four hex characters written at buf[pos..pos+3]

Example Output

- value=0x0000 -> "0000"
- value=0xFFFF -> "FFFF"
- value=0x1234 -> "1234"

Since

Version 1.0.0

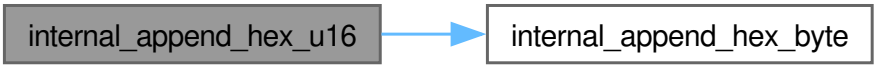
Definition at line 407 of file rx_frame_ascii.c.

```
00411 {
00412   /* Rule 5: Pre-condition invariant - callers guarantee these conditions */
00413   RX_ASSERT(buf != nullptr && max >= k_hex_chars_per_u16 && pos < max,
00414             "internal_append_hex_u16: precondition violated by caller");
00415
00416   pos = internal_append_hex_byte(buf, pos, max, (uint8_t)(value » k_u16_high_byte_shift));
00417   pos = internal_append_hex_byte(buf, pos, max, (uint8_t)(value & k_mask_u8));
00418   return pos;
00419 }
```

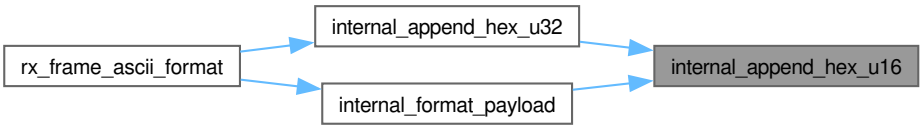
References internal_append_hex_byte(), k_hex_chars_per_u16, k_mask_u8, and k_u16_high_byte_shift.

Referenced by internal_append_hex_u32(), and internal_format_payload().

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.6 internal_append_hex_u32()

```
RX_STATIC_TESTABLE uint32_t internal_append_hex_u32 (
                char * buf,
                uint32_t pos,
                uint32_t max,
                uint32_t value)
```

Append uint32_t as eight hex characters to buffer.

Converts a 32-bit value to its eight-character hexadecimal representation (e.g., 0xDEADBEEF -> "DEADBEEF") in big-endian order. Commonly used for \rC-32 values in frame output.

Precondition

pos + 8 <= max - 1 (for writing to occur)

Postcondition

Eight hex characters written at buf[pos..pos+7]

Example Output

- value=0x00000000 -> "00000000"
- value=0xFFFFFFFF -> "FFFFFFFF"
- value=0xDEADBEEF -> "DEADBEEF"

Since

Version 1.0.0

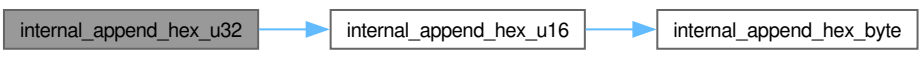
Definition at line 443 of file rx_frame_ascii.c.

```
00447 {
00448   /* Rule 5: Pre-condition invariant - callers guarantee these conditions */
00449   RX_ASSERT(buf != nullptr && max >= k_hex_chars_per_u32 && pos < max,
00450             "internal_append_hex_u32: precondition violated by caller");
00451
00452   pos = internal_append_hex_u16(buf, pos, max, (uint16_t)(value » k_u32_high_u16_shift));
00453   pos = internal_append_hex_u16(buf, pos, max, (uint16_t)(value & k_mask_u16));
00454   return pos;
00455 }
```

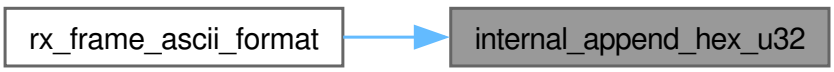
References internal_append_hex_u16(), k_hex_chars_per_u32, k_mask_u16, and k_u32_high_u16_shift.

Referenced by rx_frame_ascii_format().

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.7 internal_append_str()

```
RX_STATIC_TESTABLE uint32_t internal_append_str (
                char * buf,
                uint32_t pos,
                uint32_t max,
                const char * str)
```

Append string to buffer with bounds checking.

Copies characters from source string to output buffer until null terminator is reached or buffer is full (leaving room for final null terminator).

Precondition

pos < max (for any writing to occur)

Postcondition

Characters written in range [pos, return_value)

Buffer NOT null-terminated (caller responsibility)

Note

Does not write null terminator

Returns original pos if nothing can be written

Since

Version 1.0.0

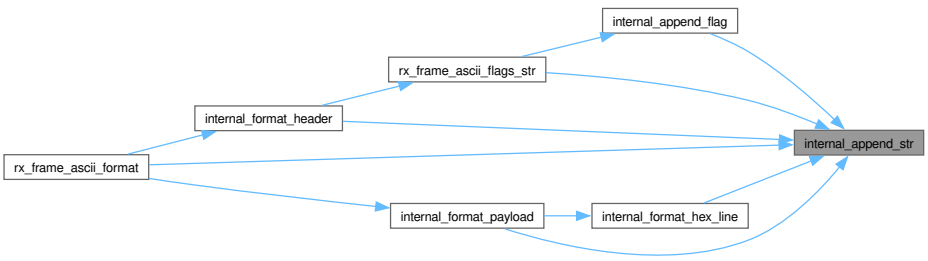
Definition at line 304 of file rx_frame_ascii.c.

```
00308 {
00309   /* Rule 5: Pre-condition invariant - callers guarantee these conditions */
00310   RX_ASSERT(buf != nullptr && str != nullptr && max != k_empty_buffer && pos < max,
00311             "internal_append_str: precondition violated by caller");
00312
00313   while (pos < max - k_null_terminator && *str != '\0') {
00314     buf[pos++] = *str++;
00315   }
00316   return pos;
00317 }
```

References k_empty_buffer, and k_null_terminator.

Referenced by internal_append_flag(), internal_format_header(), internal_format_hex_line(), internal_format_payload(), rx_frame_ascii_flags_str(), and rx_frame_ascii_format().

Here is the caller graph for this function:



4.1.2.8 internal˙format˙header()

```
RX_STATIC_TESTABLE uint32_t internal_format_header (
        char * buf,
        uint32_t pos,
        uint32_t max,
        const rx_frame_header_t * header,
        bool is_tx)
```

Format frame header metadata as single text line.

Formats the frame header fields into a human-readable summary line including direction indicator, sequence number, payload length, frame type, and flags.

Output Format

```
[TX] seq=12345 len=64 type=COMMAND flags=ACK|PRI
[RX] seq=12346 len=128 type=RESPONSE flags=NONE
```

Field Details

Field	Format	Example
Direction	[TX] or [RX]	↩ [TX]↩
Sequence	seq=N (decimal)	seq=12345
Length	len=N (decimal)	len=64
Type	type=NAME	type=COMMAND
Flags	flags=F1|F2	flags=ACK|PRI

Precondition

buf != nullptr

header != nullptr

Postcondition

Header line written with \r\n line ending

pos <= max (invariant maintained)

Since

Version 1.0.0

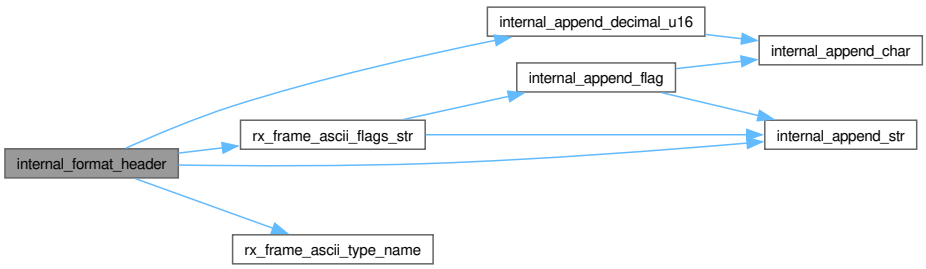
Definition at line 695 of file rx_frame_ascii.c.

```
00700 {
00701   /* Rule 5: Pre-condition invariant - callers guarantee these conditions */
00702   RX_ASSERT(buf != nullptr && header != nullptr && max != k_empty_buffer && pos < max,
00703             "internal_format_header: precondition violated by caller");
00704
00705   /* Safety guard: prevent null dereference in unit test mode after assert fires */
00706   if (header == nullptr) {
00707     return pos;
00708   }
00709
00710   char flags_buf[k_flags_buf_size];
00711
00712   /* Direction */
00713   if (is_tx) {
00714     pos = internal_append_str(buf, pos, max, "[TX] ");
00715   } else {
00716     pos = internal_append_str(buf, pos, max, "[RX] ");
00717   }
00718
00719   /* Sequence number */
00720   pos = internal_append_str(buf, pos, max, "seq=");
00721   pos = internal_append_decimal_u16(buf, pos, max, header->sequence);
00722
00723   /* Length */
00724   pos = internal_append_str(buf, pos, max, " len=");
00725   pos = internal_append_decimal_u16(buf, pos, max, header->length);
00726
00727   /* Type */
00728   pos = internal_append_str(buf, pos, max, " type=");
00729   pos = internal_append_str(buf, pos, max, rx_frame_ascii_type_name((rx_frame_type_t)header->type));
00730
00731   /* Flags */
00732   pos = internal_append_str(buf, pos, max, " flags=");
00733   (void)rx_frame_ascii_flags_str(header->flags, flags_buf, sizeof(flags_buf));
00734   pos = internal_append_str(buf, pos, max, flags_buf);
00735   pos = internal_append_str(buf, pos, max, "\r\n");
00736
00737   /* Rule 5: Post-condition: pos <= max is guaranteed by helpers' bounds checks. */
00738
00739   return pos;
00740 }
```

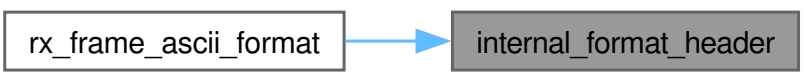
References internal_append_decimal_u16(), internal_append_str(), k_empty_buffer, k_flags_buf_size, rx_frame_ascii_flags_str(), and rx_frame_ascii_type_name().

Referenced by rx_frame_ascii_format().

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.9 internal_format_hex_line()

```
uint32_t internal_format_hex_line (
                char * buf,
                uint32_t pos,
                uint32_t max,
                const uint8_t * payload,
                uint16_t offset,
                uint16_t line_bytes)   [static]
```

Format payload as hex dump with ASCII sidebar.

Formats binary payload data as a traditional hex dump with 16 bytes per line, offset prefix, and ASCII sidebar showing printable characters (or '.' for non-printable bytes).

Output Format

Each line follows this structure:
[NNNN] XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX ....
Where:

- [NNNN] is the 4-digit hex offset from payload start
- XX XX ... is the hex representation of up to 16 bytes
- ASCII sidebar shows printable chars or '.' for non-printable

Empty Payload Handling

If length is 0, outputs: [PAYLOAD] (empty)\\r\\n

Precondition

buf != nullptr (for any output)
payload != nullptr if length > 0

Postcondition

Hex dump lines written with \r\n line endings

Loop Bounds (NASA Rule 2)

- Outer loop: bounded by length / k_bytes_per_line + 1 iterations
- Inner loops: bounded by k_bytes_per_line (16) iterations

Since

Version 1.0.0

Format one hex dump line (hex bytes + ASCII sidebar)

Since

Version 1.0.0

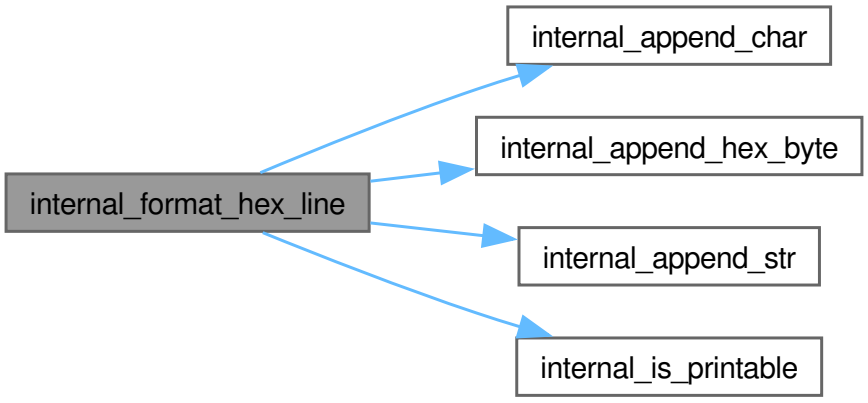
Definition at line 587 of file rx_frame_ascii.c.

```
00593 {
00594   /* Hex dump */
00595   for (uint16_t i = 0; i < k_bytes_per_line; i++) {
00596     if (i < line_bytes) {
00597       pos = internal_append_hex_byte(buf, pos, max, payload[offset + i]);
00598       pos = internal_append_char(buf, pos, max, ' ');
00599     } else {
00600       pos = internal_append_str(buf, pos, max, "   "); /* Pad short lines */
00601     }
00602   }
00603
00604   /* ASCII sidebar */
00605   pos = internal_append_char(buf, pos, max, ' ');
00606   for (uint16_t i = 0; i < line_bytes; i++) {
00607     const uint8_t c          = payload[offset + i];
00608     char          display_ch = '.';
00609     if (internal_is_printable(c)) {
00610       display_ch = (char)c;
00611     }
00612     pos = internal_append_char(buf, pos, max, display_ch);
00613   }
00614
00615   return pos;
00616 }
```

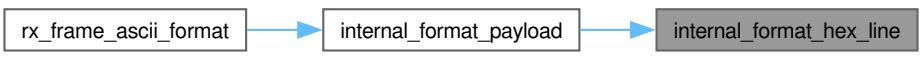
References internal_append_char(), internal_append_hex_byte(), internal_append_str(), internal_is_printable(), and k_bytes_per_line.

Referenced by internal_format_payload().

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.10 internal_is_printable()

```
RX_STATIC_TESTABLE bool internal_is_printable (
                uint8_t c)
```

Check if byte value is printable ASCII character.

Tests whether a byte value falls within the printable ASCII range (0x20-0x7E inclusive). Used to decide whether to display the actual character or a '.' placeholder in hex dump ASCII sidebars.

Printable Range

- 0x20 (space) through 0x7E (~) are printable
- 0x00-0x1F are control characters (not printable)
- 0x7F (DEL) is not printable
- 0x80-0xFF are extended ASCII (not printable in this implementation)

Note

Pure function with no side effects

Since

Version 1.0.0

Definition at line 539 of file rx_frame_ascii.c.

```
00540 {
00541   return (bool)((c >= k_printable_min) && (c <= k_printable_max));
00542 }
```

References k_printable_max, and k_printable_min.

Referenced by internal_format_hex_line().

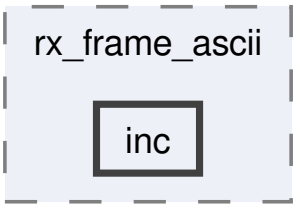
Here is the caller graph for this function:

Chapter 5

Directory Documentation

5.1 libs/rx˙frame˙ascii/inc Directory Reference

Directory dependency graph for inc:

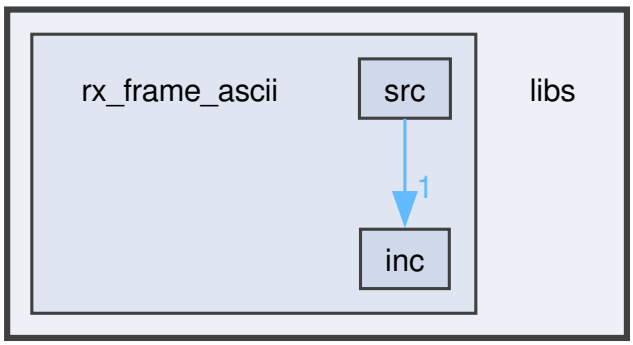


Files

- file rx_frame_ascii.h
  Frame ASCII Formatter for Human-Readable Frame Dumps.

5.2 libs Directory Reference

Directory dependency graph for libs:

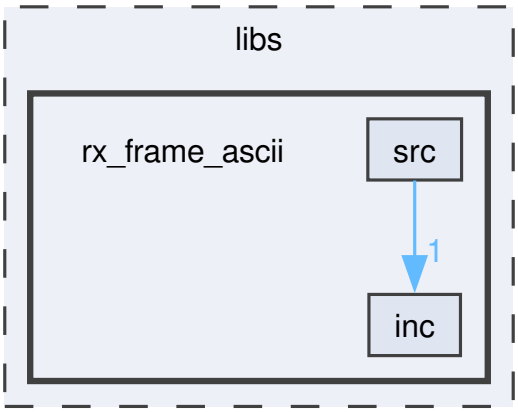


Directories

- directory rx_frame_ascii

5.3 libs/rx˙frame˙ascii Directory Reference

Directory dependency graph for rx_frame_ascii:

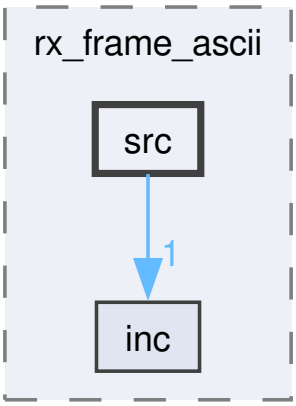


Directories

- directory inc
- directory src

5.4 libs/rx˙frame˙ascii/src Directory Reference

Directory dependency graph for src:



Files

- file rx_frame_ascii.c

  Frame ASCII Formatter Implementation.

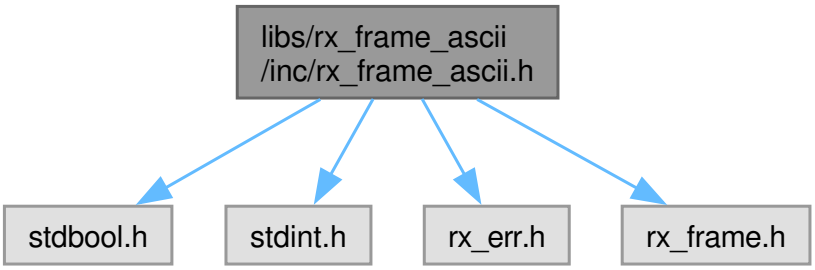
Chapter 6

File Documentation

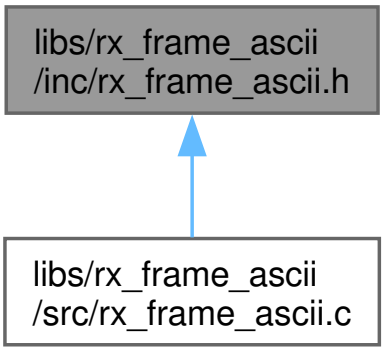
6.1 libs/rx˙frame˙ascii/inc/rx˙frame˙ascii.h File Reference

Frame ASCII Formatter for Human-Readable Frame Dumps.

#include <stdbool.h>
#include <stdint.h>
#include "rx_err.h"
#include "rx_frame.h"
Include dependency graph for rx_frame_ascii.h:



This graph shows which files directly or indirectly include this file:



Enumerations

- enum rx_frame_ascii_constants_t : uint16_t { k_frame_ascii_max_output = 2048 , k_frame_ascii_hex_per_line = 16 , k_frame_ascii_header_len = 64 , k_frame_ascii_crc_len = 32 }

  ASCII formatter configuration constants.

Functions

- rx_err_t rx_frame_ascii_format (const rx_frame_t *frame, bool is_tx, char *output, uint32_t output_max_len, uint32_t *output_len)

  Format binary frame as human-readable ASCII.
- const char * rx_frame_ascii_type_name (rx_frame_type_t type)

  Get frame type as human-readable string.
- const char * rx_frame_ascii_flags_str (uint8_t flags, char *buffer, uint32_t buffer_len)

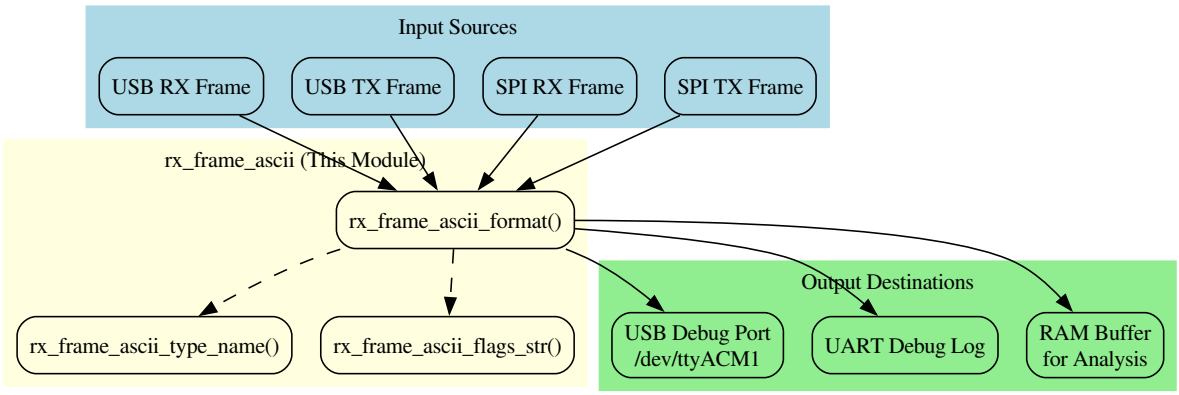
  Get frame flags as human-readable string.

6.1.1 Detailed Description

Frame ASCII Formatter for Human-Readable Frame Dumps.

Converts binary protocol frames into human-readable ASCII text for debugging and diagnostic output. This module provides a debug visualization layer that transforms the binary frame protocol (sync, headers, payload, CRC) into a structured text format suitable for terminal viewing or log analysis.

System Architecture



Output Format

The formatter produces structured text with three sections:

```
[RX] seq=1234 len=10 type=COMMAND flags=ACK|PRI
[0000] 48 65 6C 6C 6F 20 55 53 42 21 00 00 00 00 00 00  Hello USB!......
[CRC] 12345678
```

Section Details

Section	Description	Format
Header	Direction, sequence, length, type, flags	[TX/RX] seq=N len=N type=NAME flags=...
Payload	Hex dump with ASCII sidebar	[NNNN] XX XX XX... ASCII...
CRC	32-bit CRC value	[CRC] XXXXXXXX

Performance Characteristics

Metric	Value	Notes
Format time (64B payload)	∼50 us	At 240 MHz
Format time (2KB payload)	∼500 us	At 240 MHz
Stack usage	∼100 bytes	Fixed, no recursion
Output size ratio	∼4:↩ 1	Output ∼4x input size

Memory Usage

Component	Size	Description
Code size	∼2 KB	Text section
Static data	∼32 B	Hex lookup table
Stack (format)	∼100 B	Per-call stack frame
Output buffer	User-provided	Typically 2048 bytes

Design Rationale

- No dynamic allocation: All buffers provided by caller (NASA Rule 3)
- Bounded execution: All loops have static upper bounds (NASA Rule 2)
- No stdio dependency: Custom formatters avoid printf overhead and non-reentrant issues
- Hex + ASCII format: Standard debugger-style output for easy payload inspection
- Direction indicator: TX/RX prefix distinguishes send vs receive for protocol tracing

Thread Safety

All functions are thread-safe and reentrant:

- No static mutable state
- All state passed via parameters
- Pure functions (same inputs -> same outputs)

Module Dependencies

- rx_frame.h: Frame structure definitions (rx_frame_t, rx_frame_header_t)
- rx_err.h: Error code definitions
- stdint.h: Fixed-width integer types
- stdbool.h: Boolean type

NASA Power of 10 Compliance

- Rule 1: [OK] No goto, setjmp, or recursion
- Rule 2: [OK] All loops bounded by payload length or constant
- Rule 3: [OK] No dynamic memory allocation
- Rule 4: [OK] Functions are concise (<60 lines)
- Rule 5: [OK] Input validation on all public functions
- Rule 6: [OK] Variables declared at smallest scope
- Rule 7: [OK] All return values checked
- Rule 8: [OK] Constants use C23 typed enums
- Rule 9: [OK] No function pointers (pure computation)
- Rule 10: [OK] Compiles with -Wall -Wextra -Werror

SOLID Principles

- S (SRP): ASCII formatting only, no I/O operations
- O (OCP): Flag/type tables easily extended without modifying logic
- L (LSP): N/A (no inheritance hierarchy)
- I (ISP): Three focused functions for different formatting needs
- D (DIP): Depends only on frame abstractions, not concrete drivers

See also

rx_frame.h Frame structure definitions

rx_usb.h USB CDC output for debug port

docs/sections/01_nanopb_protocol.tex Protocol specification

Author

Locked, Inc.

Date

2026-01-26

Copyright

Since

Version 1.0.0

Definition in file rx_frame_ascii.h.

6.1.2 Enumeration Type Documentation

6.1.2.1 rx˙frame˙ascii˙constants˙t

enum rx_frame_ascii_constants_t : uint16_t

ASCII formatter configuration constants.

Compile-time constants for buffer sizing, line formatting, and output structure. These values are tuned for typical debugging scenarios with protobuf message payloads up to 2KB.

Buffer Sizing Rationale

- Output is approximately 4x input size due to hex expansion + ASCII sidebar
- 2048 byte max output handles ~500 byte payloads comfortably
- 16 bytes per line provides standard hex dump format (same as xxd, hexdump)

Memory Impact

These constants only affect stack usage when output buffer is provided by caller. No static buffers are allocated based on these values.

Invariant

k_frame_ascii_max_output >= 4 * k_frame_max_payload + headers

See also

rx_frame_ascii_format() Uses these constants for buffer validation

Since

Version 1.0.0

Enumerator

k_frame_ascii_max_output	Maximum recommended output buffer size in bytes. Recommended minimum buffer size for rx_frame_ascii_format() to handle maximum-size frames without truncation. Accounts for header line, hex dump lines (4 chars per byte + ASCII sidebar), and CRC footer. Value: 2048 bytes Calculation: ~500 payload x 4 expansion + 64 header + 32 CRC

k_frame_ascii_hex_per_line	Bytes per line in hex payload dump. Standard hex dump format with 16 bytes per line. Produces output like: [0000] 00 01 02 ... 0F ..... Value: 16 bytes (standard format) Note Matches xxd and hexdump default format
k_frame_ascii_header_len	Maximum header line length in characters. Buffer size for the header metadata line containing direction, sequence, length, type, and flags. Example: [RX] seq=65535 len=2048 type=COMMAND flags=ACK|PRI|RETX Value: 64 characters
k_frame_ascii_crc_len	Maximum CRC line length in characters. Buffer size for the CRC footer line. Format: [CRC] XXXXXXXX Value: 32 characters

Definition at line 178 of file rx_frame_ascii.h.

```
00178         : uint16_t {
00179     /**
00180      * @brief Maximum recommended output buffer size in bytes
00181      * @details
00182      * Recommended minimum buffer size for rx_frame_ascii_format() to handle
00183      * maximum-size frames without truncation. Accounts for header line, hex
00184      * dump lines (4 chars per byte + ASCII sidebar), and CRC footer.
00185      * @par Value: 2048 bytes
00186      * @par Calculation: ~500 payload x 4 expansion + 64 header + 32 CRC
00187      */
00188     k_frame_ascii_max_output = 2048,
00189
00190     /**
00191      * @brief Bytes per line in hex payload dump
00192      * @details
00193      * Standard hex dump format with 16 bytes per line. Produces output like:
00194      * `[0000] 00 01 02 ... 0F  .....‘
00195      * @par Value: 16 bytes (standard format)
00196      * @note Matches xxd and hexdump default format
00197      */
00198     k_frame_ascii_hex_per_line = 16,
00199
00200     /**
00201      * @brief Maximum header line length in characters
00202      * @details
00203      * Buffer size for the header metadata line containing direction, sequence,
00204      * length, type, and flags. Example: `[RX] seq=65535 len=2048 type=COMMAND flags=ACK|PRI|RETX‘
00205      * @par Value: 64 characters
00206      */
00207     k_frame_ascii_header_len = 64,
00208
00209     /**
00210      * @brief Maximum CRC line length in characters
00211      * @details
00212      * Buffer size for the CRC footer line. Format: `[CRC] XXXXXXXX‘
00213      * @par Value: 32 characters
00214      */
00215     k_frame_ascii_crc_len = 32,
00216 } rx_frame_ascii_constants_t;
```

6.1.3 Function Documentation

6.1.3.1 rx˙frame˙ascii˙flags˙str()

```
const char * rx_frame_ascii_flags_str (
                uint8_t flags,
                char * buffer,
                uint32_t buffer_len)
```

Get frame flags as human-readable string.

Formats a flags bitmask into a compact pipe-separated string showing all active flags. If no flags are set (value == 0), returns ”NONE”.

Flag Mapping

Flag Bit	String	Description
k_frame_flag_requires_ack (0x01)	”ACK”	Acknowledgment required
k_frame_flag_retransmit (0x02)	”RETX”	Retransmission
k_frame_flag_priority (0x04)	”PRI”	High priority
k_frame_flag_fec_enabled (0x08)	”FEC”	FEC encoding
k_frame_flag_soft_nack (0x10)	”SOFT”	Soft NACK
k_frame_flag_none (0x00)	”NONE”	No flags set

Output Examples

Input Flags	Output String
0x00	”NONE”
0x01	”ACK”

| 0x03 | ”ACK|RETX” | | 0x07 | ”ACK|RETX|PRI” | | 0x1F | ”ACK|RETX|PRI|FEC|SOFT” |

Parameters

in	flags	Frame flags bitmask • Value from rx_frame_flags_t or combination thereof • 0 produces ”NONE”

out	buffer	Output buffer for formatted string • Must be valid non-nullptr • Caller maintains ownership • Will be null-terminated
in	buffer_len	Size of output buffer in bytes • Minimum recommended: 32 bytes • Must be > 0

Returns

Pointer to buffer with formatted flags string

Return values

buffer	On success (contains formatted string)
""	(empty string) If buffer is nullptr or buffer_len is 0

Precondition

buffer != nullptr (or returns empty string)

buffer_len > 0 (or returns empty string)

Postcondition

buffer is null-terminated (if valid)

Return value == buffer (if valid)

Note

Thread-safe: output written to caller-provided buffer

Reentrant: no static mutable state

Warning

Buffer must be large enough for all flags + separators

Returns empty string literal "" if buffer is nullptr

Basic Usage Example

```
char flags_buf[32];
uint8_t flags = k_frame_flag_requires_ack | k_frame_flag_priority;

const char* str = rx_frame_ascii_flags_str(flags, flags_buf, sizeof(flags_buf));
// str == "ACK|PRI"
// flags_buf contains "ACK|PRI\0"
```

Error Handling Example

```
// NULL buffer returns empty string
const char* empty = rx_frame_ascii_flags_str(0x01, nullptr, 0);
// empty == ""

// Zero-length buffer returns empty string
char tiny[1];
const char* also_empty = rx_frame_ascii_flags_str(0x01, tiny, 0);
// also_empty == ""
```

Performance

- Execution time: O(n) where n = number of flag bits checked
- Stack usage: ~10 bytes

See also

rx_frame_flags_t Flag definitions

rx_frame_ascii_format() Uses this function internally

Since

Version 1.0.0

Get frame flags as human-readable string.

Implementation of rx_frame_ascii_flags_str(). Iterates through known flag bits, appending names with pipe separators.

Note

Flags formatted as pipe-separated: "ACK|RETX|PRI"

See also

rx_frame_ascii.h for full documentation

Definition at line 840 of file rx_frame_ascii.c.

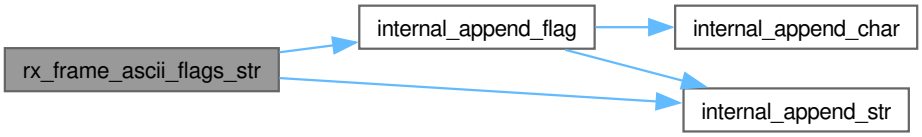
```
00841 {
00842   if (buffer == nullptr || buffer_len == k_empty_buffer) {
00843     return "";
00844   }
00845
00846   uint32_t pos = 0;
00847
00848   if (flags == k_frame_flag_none) {
00849     pos         = internal_append_str(buffer, pos, buffer_len, "NONE");
00850     buffer[pos] = '\0';
00851     return buffer;
00852   }
00853
00854   bool first = true;
00855
00856   /* Use helper to append each flag */
00857   pos =
00858     internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_requires_ack, "ACK");
00859   pos =
00860     internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_retransmit, "RETX");
00861   pos = internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_priority, "PRI");
00862   pos =
00863     internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_fec_enabled, "FEC");
00864   pos =
00865     internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_soft_nack, "SOFT");
```

```
00866
00867   buffer[pos] = '\0';
00868   return buffer;
00869 }
```

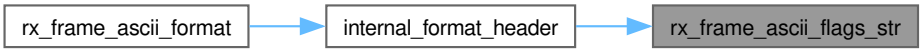
References internal_append_flag(), internal_append_str(), and k_empty_buffer.

Referenced by internal_format_header().

Here is the call graph for this function:



Here is the caller graph for this function:



6.1.3.2 rx_frame_ascii_format()

```
rx_err_t rx_frame_ascii_format (
                const rx_frame_t * frame,
                bool is_tx,
                char * output,
                uint32_t output_max_len,
                uint32_t * output_len)   [nodiscard]
```

Format binary frame as human-readable ASCII.

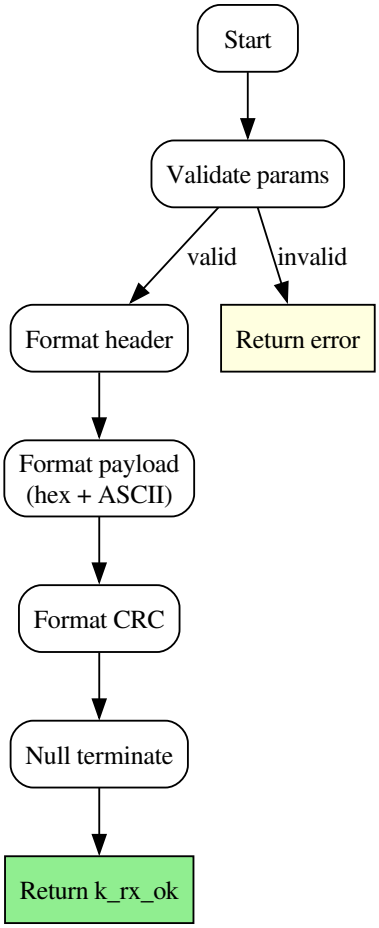
Converts a complete frame structure into a formatted ASCII string suitable for debugging output, logging, or terminal display. The output is structured in three sections: header metadata, hex payload dump with ASCII sidebar, and CRC footer.

Algorithm Steps

1. Validate all input pointers and buffer size
2. Format header line: direction, sequence, length, type, flags
3. Format payload as hex dump with 16 bytes per line + ASCII sidebar
4. Format CRC footer with 8-digit hex value
5. Null-terminate output and return length

Output Structure

```
[TX] seq=1234 len=10 type=COMMAND flags=ACK|PRI
[0000] 48 65 6C 6C 6F 20 55 53 42 21 00 00 00 00 00 00  Hello USB!......
[CRC] 12345678
```



Parameters

in	frame	Frame to format • Must be valid non-nullptr • header.length must be <= k_frame_max_payload • payload data must be valid for header.length bytes
in	is_tx	Direction indicator • true: Prefixes output with "[TX] " (transmit direction) • false: Prefixes output with "[RX] " (receive direction)

out	output	Output buffer for ASCII text • Must be valid non-nullptr • Caller maintains ownership • Will be null-terminated on success
in	output_max_len	Maximum bytes to write to output buffer • Must be >= 64 (minimum for header + CRC with empty payload) • Recommended: k_frame_ascii_max_output (2048) • Larger payloads require proportionally larger buffers
out	output_len	Actual bytes written (excluding null terminator) • Must be valid non-nullptr • Set only on success

Returns

rx_err_t Error code

Return values

k_rx_ok	Frame formatted successfully
k_rx_err_invalid_arg	frame, output, or output_len is nullptr, or frame->header.length > k_frame_max_payload
k_rx_err_no_mem	Output buffer too small (< 64 bytes or truncation occurred)

Precondition

frame != nullptr

output != nullptr

output_len != nullptr

output_max_len >= 64

frame->header.length <= k_frame_max_payload

Postcondition

output is null-terminated

*output_len contains actual output length (on success)

output buffer contains formatted ASCII text (on success)

Invariant

Output never exceeds output_max_len bytes

Note

Thread-safe: no static mutable state

Reentrant: all state passed via parameters

Warning

Truncation returns k_rx_err_no_mem; output may be partial

Empty payload (length=0) produces [PAYLOAD] (empty) line

Basic Usage Example

```
rx_frame_t frame;
char output[2048];
uint32_t output_len;

// Receive a frame from USB or SPI
rx_usb_comm_receive(&handle, &frame, 100);

// Format for debug output
rx_err_t err = rx_frame_ascii_format(&frame, false, output, sizeof(output), &output_len);
if (err == k_rx_ok) {
    // Send to debug USB port
    rx_usb_write(k_usb_port_debug, (const uint8_t*)output, output_len);
}
```

Error Handling Example

```
rx_err_t err = rx_frame_ascii_format(&frame, true, buf, sizeof(buf), &len);
switch (err) {
    case k_rx_ok:
        // Success - output contains formatted frame
        break;
    case k_rx_err_invalid_arg:
        rx_log_error("ASCII", "Invalid frame or buffer pointer");
        break;
    case k_rx_err_no_mem:
        rx_log_warn("ASCII", "Buffer too small, output truncated");
        break;
    default:
        rx_log_error("ASCII", "Unexpected error");
        break;
}
```

Performance

- Execution time: ~50 us for 64-byte payload @ 240 MHz
- Stack usage: ~100 bytes (fixed)
- No heap allocation

See also

rx_frame_ascii_type_name() Convert type enum to string

rx_frame_ascii_flags_str() Convert flags to string

rx_frame_t Frame structure definition

Since

Version 1.0.0

Format binary frame as human-readable ASCII.

Implementation of rx_frame_ascii_format(). Orchestrates header, payload, and \rC formatting using internal helper functions.

Implementation Notes

- Validates all inputs before any output
- Calls internal_format_header() for metadata line
- Calls internal_format_payload() for hex dump
- Appends \rC footer directly
- Detects truncation and returns k_rx_err_no_mem

See also

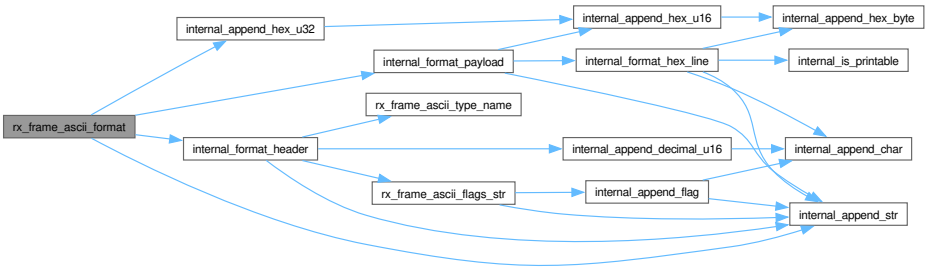
rx_frame_ascii.h for full documentation

Definition at line 889 of file rx_frame_ascii.c.

```
00894 {
00895   /* Rule 5: Pre-condition validation */
00896   if (frame == nullptr) {
00897     return k_rx_err_invalid_arg;
00898   }
00899   if (output == nullptr) {
00900     return k_rx_err_invalid_arg;
00901   }
00902   if (output_len == nullptr) {
00903     return k_rx_err_invalid_arg;
00904   }
00905   if (output_max_len < k_min_output_size) {
00906     return k_rx_err_no_mem;
00907   }
00908
00909   /* Validate payload length does not exceed frame capacity */
00910   if (frame->header.length > k_frame_max_payload) {
00911     return k_rx_err_invalid_arg;
00912   }
00913
00914   uint32_t pos = 0;
00915
00916   /* Format header metadata */
00917   pos = internal_format_header(output, pos, output_max_len, &frame->header, is_tx);
00918
00919   /* Payload hex dump - use validated length to prevent out-of-bounds reads */
00920   pos = internal_format_payload(output, pos, output_max_len, frame->payload, frame->header.length);
00921
00922   /* CRC */
00923   pos = internal_append_str(output, pos, output_max_len, "[CRC] ");
00924   pos = internal_append_hex_u32(output, pos, output_max_len, frame->crc);
00925   pos = internal_append_str(output, pos, output_max_len, "\r\n");
00926
00927   /* Rule 5: Post-condition: helpers cap pos at output_max_len - 1 via bounds
00928    * checks, so pos < output_max_len is an invariant by construction. */
00929
00930   /* Null terminate */
00931   output[pos] = '\0';
00932
00933   *output_len = pos;
00934
00935   return k_rx_ok;
00936 }
```

References internal_append_hex_u32(), internal_append_str(), internal_format_header(), internal_format_payload(), and k_min_output_size.

Here is the call graph for this function:



6.1.3.3 rx˙frame˙ascii˙type˙name()

const char * rx_frame_ascii_type_name (
rx_frame_type_t type)

Get frame type as human-readable string.

Converts a frame type enumeration value to its corresponding string name for display or logging. Returns a static string literal that does not require freeing.

Type Mapping

Type Value	String Output
k_frame_type_ping (0x00)	"PING"
k_frame_type_pong (0x01)	"PONG"
k_frame_type_command (0x10)	"COMMAND"
k_frame_type_response (0x11)	"RESPONSE"
k_frame_type_ack (0x12)	"ACK"
k_frame_type_nack (0x13)	"NACK"
k_frame_type_reset_ack (0xFE)	"RESET_ACK"
k_frame_type_reset (0xFF)	"RESET"
(any other value)	"UNKNOWN"

Parameters

in	type	Frame type enumeration value • Any rx_frame_type_t value • Unknown values return "UNKNOWN"

Returns

Pointer to static string literal

Return values

PING	For k_frame_type_ping
PONG	For k_frame_type_pong
COMMAND	For k_frame_type_command
RESPONSE	For k_frame_type_response
ACK	For k_frame_type_ack
NACK	For k_frame_type_nack
RESET_ACK	For k_frame_type_reset_ack
RESET	For k_frame_type_reset
UNKNOWN	For unrecognized values

Precondition

None (pure function, always valid)

Postcondition

Return value is non-nullptr to null-terminated string

Return value points to static read-only memory

Note

Thread-safe: returns pointer to static const data

Reentrant: no mutable state

Do NOT free the returned pointer

Example

```
rx_frame_type_t type = k_frame_type_command;
const char* name = rx_frame_ascii_type_name(type);
// name == "COMMAND"

// Safe with invalid values
const char* unknown = rx_frame_ascii_type_name((rx_frame_type_t)0xFF);
// unknown == "UNKNOWN"
```

Performance

- Execution time: O(1) - switch statement
- Stack usage: 0 bytes additional

See also

rx_frame_type_t Frame type enumeration

rx_frame_ascii_format() Uses this function internally

Since

Version 1.0.0

Get frame type as human-readable string.

Implementation of rx_frame_ascii_type_name(). Uses switch statement for O(1) lookup with compile-time string literals.

See also

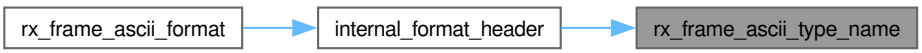
rx_frame_ascii.h for full documentation

Definition at line 758 of file rx_frame_ascii.c.

```
00759 {
00760   switch (type) {
00761     case k_frame_type_ping:
00762       return "PING";
00763     case k_frame_type_pong:
00764       return "PONG";
00765     case k_frame_type_command:
00766       return "COMMAND";
00767     case k_frame_type_response:
00768       return "RESPONSE";
00769     case k_frame_type_ack:
00770       return "ACK";
00771     case k_frame_type_nack:
00772       return "NACK";
00773     case k_frame_type_log_message:
00774       return "LOG_MESSAGE";
00775     case k_frame_type_reset_ack:
00776       return "RESET_ACK";
00777     case k_frame_type_reset:
00778       return "RESET";
00779     default:
00780       return "UNKNOWN";
00781   }
00782 }
```

Referenced by internal_format_header().

Here is the caller graph for this function:



6.2 rx˙frame˙ascii.h

Go to the documentation of this file.

```
00001 /**
00002  * @file rx_frame_ascii.h
00003  * @brief Frame ASCII Formatter for Human-Readable Frame Dumps
00004  *
00005  * @details
00006  * Converts binary protocol frames into human-readable ASCII text for debugging
00007  * and diagnostic output. This module provides a debug visualization layer that
00008  * transforms the binary frame protocol (sync, headers, payload, CRC) into a
00009  * structured text format suitable for terminal viewing or log analysis.
00010  *
00011  * @par System Architecture
00012  * @dot
00013  * digraph frame_ascii_architecture {
00014  *   rankdir=TB;
00015  *   node [shape=box, style=rounded];
00016  *
00017  *   subgraph cluster_input {
00018  *     label="Input Sources";
00019  *     style=filled;
```

```
00020 *     color=lightblue;
00021 *     usb_rx [label="USB RX Frame"];
00022 *     usb_tx [label="USB TX Frame"];
00023 *     spi_rx [label="SPI RX Frame"];
00024 *     spi_tx [label="SPI TX Frame"];
00025 *   }
00026 *
00027 *   subgraph cluster_formatter {
00028 *     label="rx_frame_ascii (This Module)";
00029 *     style=filled;
00030 *     color=lightyellow;
00031 *     format [label="rx_frame_ascii_format()"];
00032 *     type_name [label="rx_frame_ascii_type_name()"];
00033 *     flags_str [label="rx_frame_ascii_flags_str()"];
00034 *   }
00035 *
00036 *   subgraph cluster_output {
00037 *     label="Output Destinations";
00038 *     style=filled;
00039 *     color=lightgreen;
00040 *     usb_debug [label="USB Debug Port\n/dev/ttyACM1"];
00041 *     uart_log [label="UART Debug Log"];
00042 *     buffer [label="RAM Buffer\nfor Analysis"];
00043 *   }
00044 *
00045 *   usb_rx -> format;
00046 *   usb_tx -> format;
00047 *   spi_rx -> format;
00048 *   spi_tx -> format;
00049 *   format -> usb_debug;
00050 *   format -> uart_log;
00051 *   format -> buffer;
00052 *   format -> type_name [style=dashed];
00053 *   format -> flags_str [style=dashed];
00054 * }
00055 * @enddot
00056 *
00057 * @par Output Format
00058 * The formatter produces structured text with three sections:
00059 * ```
00060 * [RX] seq=1234 len=10 type=COMMAND flags=ACK|PRI
00061 * [0000] 48 65 6C 6C 6F 20 55 53 42 21 00 00 00 00 00 00  Hello USB!......
00062 * [CRC] 12345678
00063 * ```
00064 *
00065 * @par Section Details
00066 * | Section | Description | Format |
00067 * |---------|-------------|--------|
00068 * | Header  | Direction, sequence, length, type, flags | `[TX/RX] seq=N len=N type=NAME flags=...` |
00069 * | Payload | Hex dump with ASCII sidebar | `[NNNN] XX XX XX...  ASCII...` |
00070 * | CRC     | 32-bit CRC value | `[CRC] XXXXXXXX` |
00071 *
00072 * @par Performance Characteristics
00073 * | Metric | Value | Notes |
00074 * |--------|-------|-------|
00075 * | Format time (64B payload) | ~50 us | At 240 MHz |
00076 * | Format time (2KB payload) | ~500 us | At 240 MHz |
00077 * | Stack usage | ~100 bytes | Fixed, no recursion |
00078 * | Output size ratio | ~4:1 | Output ~4x input size |
00079 *
00080 * @par Memory Usage
00081 * | Component | Size | Description |
00082 * |-----------|------|-------------|
00083 * | Code size | ~2 KB | Text section |
00084 * | Static data | ~32 B | Hex lookup table |
00085 * | Stack (format) | ~100 B | Per-call stack frame |
00086 * | Output buffer | User-provided | Typically 2048 bytes |
00087 *
00088 * @par Design Rationale
00089 * - **No dynamic allocation**: All buffers provided by caller (NASA Rule 3)
00090 * - **Bounded execution**: All loops have static upper bounds (NASA Rule 2)
00091 * - **No stdio dependency**: Custom formatters avoid printf overhead and non-reentrant issues
00092 * - **Hex + ASCII format**: Standard debugger-style output for easy payload inspection
00093 * - **Direction indicator**: TX/RX prefix distinguishes send vs receive for protocol tracing
00094 *
00095 * @par Thread Safety
00096 * All functions are **thread-safe** and **reentrant**:
00097 * - No static mutable state
00098 * - All state passed via parameters
00099 * - Pure functions (same inputs -> same outputs)
00100 *
00101 * @par Module Dependencies
00102 * - rx_frame.h: Frame structure definitions (rx_frame_t, rx_frame_header_t)
00103 * - rx_err.h: Error code definitions
00104 * - stdint.h: Fixed-width integer types
00105 * - stdbool.h: Boolean type
00106 *
```

```
00107  * @par NASA Power of 10 Compliance
00108  * - Rule 1: [OK] No goto, setjmp, or recursion
00109  * - Rule 2: [OK] All loops bounded by payload length or constant
00110  * - Rule 3: [OK] No dynamic memory allocation
00111  * - Rule 4: [OK] Functions are concise (<60 lines)
00112  * - Rule 5: [OK] Input validation on all public functions
00113  * - Rule 6: [OK] Variables declared at smallest scope
00114  * - Rule 7: [OK] All return values checked
00115  * - Rule 8: [OK] Constants use C23 typed enums
00116  * - Rule 9: [OK] No function pointers (pure computation)
00117  * - Rule 10: [OK] Compiles with -Wall -Wextra -Werror
00118  *
00119  * @par SOLID Principles
00120  * - **S (SRP):** ASCII formatting only, no I/O operations
00121  * - **O (OCP):** Flag/type tables easily extended without modifying logic
00122  * - **L (LSP):** N/A (no inheritance hierarchy)
00123  * - **I (ISP):** Three focused functions for different formatting needs
00124  * - **D (DIP):** Depends only on frame abstractions, not concrete drivers
00125  *
00126  * @see rx_frame.h Frame structure definitions
00127  * @see rx_usb.h USB CDC output for debug port
00128  * @see docs/sections/01_nanopb_protocol.tex Protocol specification
00129  *
00130  * @author Locked, Inc.
00131  * @date 2026-01-26
00132  * @copyright Copyright (c) 2026 Locked Inc.
00133  * SPDX-License-Identifier: MIT
00134  * @since Version 1.0.0
00135  */
00136
00137 #pragma once
00138
00139 #include <stdbool.h>
00140 #include <stdint.h>
00141
00142 #include "rx_err.h"
00143 #include "rx_frame.h"
00144
00145 #ifdef __cplusplus
00146 extern "C" {
00147 #endif
00148
00149 /*
      ================================================================================
00150  * Configuration Constants
00151  *
      ================================================================================
00152  */
00153
00154 /**
00155  * @enum rx_frame_ascii_constants_t
00156  * @brief ASCII formatter configuration constants
00157  *
00158  * @details
00159  * Compile-time constants for buffer sizing, line formatting, and output
00160  * structure. These values are tuned for typical debugging scenarios with
00161  * protobuf message payloads up to 2KB.
00162  *
00163  * @par Buffer Sizing Rationale
00164  * - Output is approximately 4x input size due to hex expansion + ASCII sidebar
00165  * - 2048 byte max output handles ~500 byte payloads comfortably
00166  * - 16 bytes per line provides standard hex dump format (same as xxd, hexdump)
00167  *
00168  * @par Memory Impact
00169  * These constants only affect stack usage when output buffer is provided by
00170  * caller. No static buffers are allocated based on these values.
00171  *
00172  * @invariant k_frame_ascii_max_output >= 4 * k_frame_max_payload + headers
00173  *
00174  * @see rx_frame_ascii_format() Uses these constants for buffer validation
00175  *
00176  * @since Version 1.0.0
00177  */
00178 typedef enum : uint16_t {
00179     /**
00180      * @brief Maximum recommended output buffer size in bytes
00181      * @details
00182      * Recommended minimum buffer size for rx_frame_ascii_format() to handle
00183      * maximum-size frames without truncation. Accounts for header line, hex
00184      * dump lines (4 chars per byte + ASCII sidebar), and CRC footer.
00185      * @par Value: 2048 bytes
00186      * @par Calculation: ~500 payload x 4 expansion + 64 header + 32 CRC
00187      */
00188     k_frame_ascii_max_output = 2048,
00189
00190     /**
00191      * @brief Bytes per line in hex payload dump
```

```
00192     * @details
00193     * Standard hex dump format with 16 bytes per line. Produces output like:
00194     * `[0000] 00 01 02 ... 0F  ................`
00195     * @par Value: 16 bytes (standard format)
00196     * @note Matches xxd and hexdump default format
00197     */
00198    k_frame_ascii_hex_per_line = 16,
00199
00200    /**
00201     * @brief Maximum header line length in characters
00202     * @details
00203     * Buffer size for the header metadata line containing direction, sequence,
00204     * length, type, and flags. Example: `[RX] seq=65535 len=2048 type=COMMAND flags=ACK|PRI|RETX`
00205     * @par Value: 64 characters
00206     */
00207    k_frame_ascii_header_len = 64,
00208
00209    /**
00210     * @brief Maximum CRC line length in characters
00211     * @details
00212     * Buffer size for the CRC footer line. Format: `[CRC] XXXXXXXX`
00213     * @par Value: 32 characters
00214     */
00215    k_frame_ascii_crc_len = 32,
00216 } rx_frame_ascii_constants_t;
00217
00218 /*
    ============================================================================
00219 * Public API
00220 *
    ============================================================================
00221 */
00222
00223 /**
00224 * @brief Format binary frame as human-readable ASCII
00225 *
00226 * @details
00227 * Converts a complete frame structure into a formatted ASCII string suitable
00228 * for debugging output, logging, or terminal display. The output is structured
00229 * in three sections: header metadata, hex payload dump with ASCII sidebar, and
00230 * CRC footer.
00231 *
00232 * @par Algorithm Steps
00233 * 1. Validate all input pointers and buffer size
00234 * 2. Format header line: direction, sequence, length, type, flags
00235 * 3. Format payload as hex dump with 16 bytes per line + ASCII sidebar
00236 * 4. Format CRC footer with 8-digit hex value
00237 * 5. Null-terminate output and return length
00238 *
00239 * @par Output Structure
00240 * ```
00241 * [TX] seq=1234 len=10 type=COMMAND flags=ACK|PRI
00242 * [0000] 48 65 6C 6C 6F 20 55 53 42 21 00 00 00 00 00 00  Hello USB!......
00243 * [CRC] 12345678
00244 * ```
00245 *
00246 * @dot
00247 * digraph format_flow {
00248 *   rankdir=TB;
00249 *   node [shape=box, style=rounded];
00250 *   start [label="Start"];
00251 *   validate [label="Validate params"];
00252 *   header [label="Format header"];
00253 *   payload [label="Format payload\n(hex + ASCII)"];
00254 *   crc [label="Format CRC"];
00255 *   terminate [label="Null terminate"];
00256 *   done [label="Return k_rx_ok" style=filled, fillcolor=lightgreen];
00257 *   error [label="Return error" style=filled, fillcolor=lightyellow];
00258 *   start -> validate;
00259 *   validate -> header [label="valid"];
00260 *   validate -> error [label="invalid"];
00261 *   header -> payload;
00262 *   payload -> crc;
00263 *   crc -> terminate;
00264 *   terminate -> done;
00265 * }
00266 * @enddot
00267 *
00268 * @param[in] frame Frame to format
00269 *   - Must be valid non-nullptr
00270 *   - header.length must be <= k_frame_max_payload
00271 *   - payload data must be valid for header.length bytes
00272 * @param[in] is_tx Direction indicator
00273 *   - true: Prefixes output with "[TX] " (transmit direction)
00274 *   - false: Prefixes output with "[RX] " (receive direction)
00275 * @param[out] output Output buffer for ASCII text
00276 *   - Must be valid non-nullptr
```

```
00277  *   - Caller maintains ownership
00278  *   - Will be null-terminated on success
00279  * @param[in] output_max_len Maximum bytes to write to output buffer
00280  *   - Must be >= 64 (minimum for header + CRC with empty payload)
00281  *   - Recommended: k_frame_ascii_max_output (2048)
00282  *   - Larger payloads require proportionally larger buffers
00283  * @param[out] output_len Actual bytes written (excluding null terminator)
00284  *   - Must be valid non-nullptr
00285  *   - Set only on success
00286  *
00287  * @return rx_err_t Error code
00288  * @retval k_rx_ok Frame formatted successfully
00289  * @retval k_rx_err_invalid_arg frame, output, or output_len is nullptr,
00290  *         or frame->header.length > k_frame_max_payload
00291  * @retval k_rx_err_no_mem Output buffer too small (< 64 bytes or truncation occurred)
00292  *
00293  * @pre frame != nullptr
00294  * @pre output != nullptr
00295  * @pre output_len != nullptr
00296  * @pre output_max_len >= 64
00297  * @pre frame->header.length <= k_frame_max_payload
00298  *
00299  * @post output is null-terminated
00300  * @post *output_len contains actual output length (on success)
00301  * @post output buffer contains formatted ASCII text (on success)
00302  *
00303  * @invariant Output never exceeds output_max_len bytes
00304  *
00305  * @note Thread-safe: no static mutable state
00306  * @note Reentrant: all state passed via parameters
00307  *
00308  * @warning Truncation returns k_rx_err_no_mem; output may be partial
00309  * @warning Empty payload (length=0) produces `[PAYLOAD] (empty)‘ line
00310  *
00311  * @par Basic Usage Example
00312  * @code
00313  * rx_frame_t frame;
00314  * char output[2048];
00315  * uint32_t output_len;
00316  *
00317  * // Receive a frame from USB or SPI
00318  * rx_usb_comm_receive(&handle, &frame, 100);
00319  *
00320  * // Format for debug output
00321  * rx_err_t err = rx_frame_ascii_format(&frame, false, output, sizeof(output), &output_len);
00322  * if (err == k_rx_ok) {
00323  *     // Send to debug USB port
00324  *     rx_usb_write(k_usb_port_debug, (const uint8_t*)output, output_len);
00325  * }
00326  * @endcode
00327  *
00328  * @par Error Handling Example
00329  * @code
00330  * rx_err_t err = rx_frame_ascii_format(&frame, true, buf, sizeof(buf), &len);
00331  * switch (err) {
00332  *     case k_rx_ok:
00333  *         // Success - output contains formatted frame
00334  *         break;
00335  *     case k_rx_err_invalid_arg:
00336  *         rx_log_error("ASCII", "Invalid frame or buffer pointer");
00337  *         break;
00338  *     case k_rx_err_no_mem:
00339  *         rx_log_warn("ASCII", "Buffer too small, output truncated");
00340  *         break;
00341  *     default:
00342  *         rx_log_error("ASCII", "Unexpected error");
00343  *         break;
00344  * }
00345  * @endcode
00346  *
00347  * @par Performance
00348  * - Execution time: ~50 us for 64-byte payload @ 240 MHz
00349  * - Stack usage: ~100 bytes (fixed)
00350  * - No heap allocation
00351  *
00352  * @see rx_frame_ascii_type_name() Convert type enum to string
00353  * @see rx_frame_ascii_flags_str() Convert flags to string
00354  * @see rx_frame_t Frame structure definition
00355  *
00356  * @since Version 1.0.0
00357  */
00358 [[nodiscard]] rx_err_t rx_frame_ascii_format(const rx_frame_t* frame,
00359                                              bool             is_tx,
00360                                              char*            output,
00361                                              uint32_t         output_max_len,
00362                                              uint32_t*        output_len);
00363
```

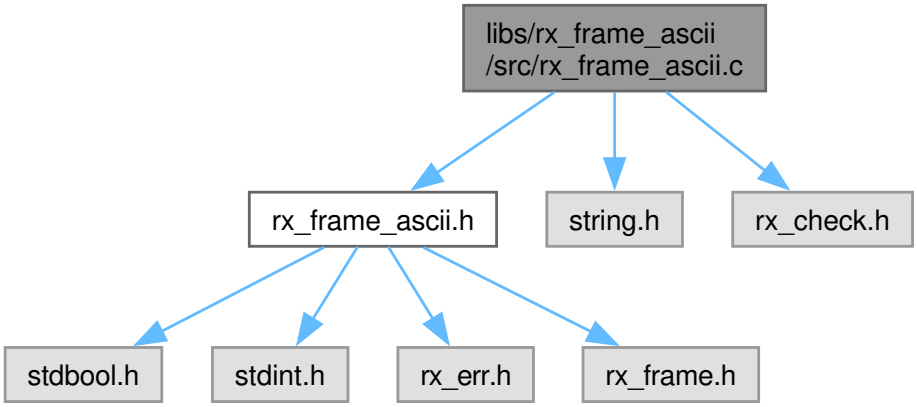
```
00364 /**
00365  * @brief Get frame type as human-readable string
00366  *
00367  * @details
00368  * Converts a frame type enumeration value to its corresponding string name
00369  * for display or logging. Returns a static string literal that does not
00370  * require freeing.
00371  *
00372  * @par Type Mapping
00373  * | Type Value | String Output |
00374  * |------------|---------------|
00375  * | k_frame_type_ping (0x00) | "PING" |
00376  * | k_frame_type_pong (0x01) | "PONG" |
00377  * | k_frame_type_command (0x10) | "COMMAND" |
00378  * | k_frame_type_response (0x11) | "RESPONSE" |
00379  * | k_frame_type_ack (0x12) | "ACK" |
00380  * | k_frame_type_nack (0x13) | "NACK" |
00381  * | k_frame_type_reset_ack (0xFE) | "RESET_ACK" |
00382  * | k_frame_type_reset (0xFF) | "RESET" |
00383  * | (any other value) | "UNKNOWN" |
00384  *
00385  * @param[in] type Frame type enumeration value
00386  *   - Any rx_frame_type_t value
00387  *   - Unknown values return "UNKNOWN"
00388  *
00389  * @return Pointer to static string literal
00390  * @retval "PING" For k_frame_type_ping
00391  * @retval "PONG" For k_frame_type_pong
00392  * @retval "COMMAND" For k_frame_type_command
00393  * @retval "RESPONSE" For k_frame_type_response
00394  * @retval "ACK" For k_frame_type_ack
00395  * @retval "NACK" For k_frame_type_nack
00396  * @retval "RESET_ACK" For k_frame_type_reset_ack
00397  * @retval "RESET" For k_frame_type_reset
00398  * @retval "UNKNOWN" For unrecognized values
00399  *
00400  * @pre None (pure function, always valid)
00401  *
00402  * @post Return value is non-nullptr to null-terminated string
00403  * @post Return value points to static read-only memory
00404  *
00405  * @note Thread-safe: returns pointer to static const data
00406  * @note Reentrant: no mutable state
00407  * @note Do NOT free the returned pointer
00408  *
00409  * @par Example
00410  * @code
00411  * rx_frame_type_t type = k_frame_type_command;
00412  * const char* name = rx_frame_ascii_type_name(type);
00413  * // name == "COMMAND"
00414  *
00415  * // Safe with invalid values
00416  * const char* unknown = rx_frame_ascii_type_name((rx_frame_type_t)0xFF);
00417  * // unknown == "UNKNOWN"
00418  * @endcode
00419  *
00420  * @par Performance
00421  * - Execution time: O(1) - switch statement
00422  * - Stack usage: 0 bytes additional
00423  *
00424  * @see rx_frame_type_t Frame type enumeration
00425  * @see rx_frame_ascii_format() Uses this function internally
00426  *
00427  * @since Version 1.0.0
00428  */
00429 const char* rx_frame_ascii_type_name(rx_frame_type_t type);
00430
00431 /**
00432  * @brief Get frame flags as human-readable string
00433  *
00434  * @details
00435  * Formats a flags bitmask into a compact pipe-separated string showing all
00436  * active flags. If no flags are set (value == 0), returns "NONE".
00437  *
00438  * @par Flag Mapping
00439  * | Flag Bit | String | Description |
00440  * |----------|--------|-------------|
00441  * | k_frame_flag_requires_ack (0x01) | "ACK" | Acknowledgment required |
00442  * | k_frame_flag_retransmit (0x02) | "RETX" | Retransmission |
00443  * | k_frame_flag_priority (0x04) | "PRI" | High priority |
00444  * | k_frame_flag_fec_enabled (0x08) | "FEC" | FEC encoding |
00445  * | k_frame_flag_soft_nack (0x10) | "SOFT" | Soft NACK |
00446  * | k_frame_flag_none (0x00) | "NONE" | No flags set |
00447  *
00448  * @par Output Examples
00449  * | Input Flags | Output String |
00450  * |-------------|---------------|
```

```
00451  * | 0x00 | "NONE" |
00452  * | 0x01 | "ACK" |
00453  * | 0x03 | "ACK|RETX" |
00454  * | 0x07 | "ACK|RETX|PRI" |
00455  * | 0x1F | "ACK|RETX|PRI|FEC|SOFT" |
00456  *
00457  * @param[in] flags Frame flags bitmask
00458  *   - Value from rx_frame_flags_t or combination thereof
00459  *   - 0 produces "NONE"
00460  * @param[out] buffer Output buffer for formatted string
00461  *   - Must be valid non-nullptr
00462  *   - Caller maintains ownership
00463  *   - Will be null-terminated
00464  * @param[in] buffer_len Size of output buffer in bytes
00465  *   - Minimum recommended: 32 bytes
00466  *   - Must be > 0
00467  *
00468  * @return Pointer to buffer with formatted flags string
00469  * @retval buffer On success (contains formatted string)
00470  * @retval "" (empty string) If buffer is nullptr or buffer_len is 0
00471  *
00472  * @pre buffer != nullptr (or returns empty string)
00473  * @pre buffer_len > 0 (or returns empty string)
00474  *
00475  * @post buffer is null-terminated (if valid)
00476  * @post Return value == buffer (if valid)
00477  *
00478  * @note Thread-safe: output written to caller-provided buffer
00479  * @note Reentrant: no static mutable state
00480  *
00481  * @warning Buffer must be large enough for all flags + separators
00482  * @warning Returns empty string literal "" if buffer is nullptr
00483  *
00484  * @par Basic Usage Example
00485  * @code
00486  * char flags_buf[32];
00487  * uint8_t flags = k_frame_flag_requires_ack | k_frame_flag_priority;
00488  *
00489  * const char* str = rx_frame_ascii_flags_str(flags, flags_buf, sizeof(flags_buf));
00490  * // str == "ACK|PRI"
00491  * // flags_buf contains "ACK|PRI\0"
00492  * @endcode
00493  *
00494  * @par Error Handling Example
00495  * @code
00496  * // NULL buffer returns empty string
00497  * const char* empty = rx_frame_ascii_flags_str(0x01, nullptr, 0);
00498  * // empty == ""
00499  *
00500  * // Zero-length buffer returns empty string
00501  * char tiny[1];
00502  * const char* also_empty = rx_frame_ascii_flags_str(0x01, tiny, 0);
00503  * // also_empty == ""
00504  * @endcode
00505  *
00506  * @par Performance
00507  * - Execution time: O(n) where n = number of flag bits checked
00508  * - Stack usage: ~10 bytes
00509  *
00510  * @see rx_frame_flags_t Flag definitions
00511  * @see rx_frame_ascii_format() Uses this function internally
00512  *
00513  * @since Version 1.0.0
00514  */
00515 const char* rx_frame_ascii_flags_str(uint8_t flags, char* buffer, uint32_t buffer_len);
00516
00517 #ifdef ___cplusplus
00518 }
00519 #endif
```

6.3 libs/rx˙frame˙ascii/src/rx˙frame˙ascii.c File Reference

Frame ASCII Formatter Implementation.

#include "rx_frame_ascii.h"
#include <string.h>
#include "rx_check.h"
Include dependency graph for rx_frame_ascii.c:



Enumerations

- enum format_constants_t : uint16_t { k_hex_digit_shift = 4 , k_hex_digit_mask = 0xF , k_printable_min = 0x20 , k_printable_max = 0x7E , k_bytes_per_line = 16 , k_min_output_size = 64 , k_hex_space_gap = 2 , k_mask_u8 = 0xFF , k_mask_u16 = 0xFFFF , k_decimal_base = 10 , k_u16_decimal_max_digits = 5 , k_flags_buf_size = 32 }

  String formatting constants for hex and decimal conversion.
- enum buffer_size_constants_t : uint8_t { k_empty_buffer = 0 , k_null_terminator = 1 , k_hex_chars_per_byte = 2 , k_hex_chars_per_u16 = 4 , k_hex_chars_per_u32 = 8 , k_u16_high_byte_shift = 8 , k_u32_high_u16_shift = 16 }

  Buffer size and offset constants for string building.
- enum decimal_buf_constants_t : uint8_t { k_u16_decimal_buf_size = k_u16_decimal_max_digits + 1 }

  Decimal conversion buffer size constants.

Functions

- RX_STATIC_TESTABLE uint32_t internal_append_str (char *buf, uint32_t pos, uint32_t max, const char *str)

  Append string to buffer with bounds checking.
- RX_STATIC_TESTABLE uint32_t internal_append_char (char *buf, uint32_t pos, uint32_t max, char c)

  Append single character to buffer with bounds checking.
- RX_STATIC_TESTABLE uint32_t internal_append_hex_byte (char *buf, uint32_t pos, uint32_t max, uint8_t byte)

  Append byte as two hex characters to buffer.
- RX_STATIC_TESTABLE uint32_t internal_append_hex_u16 (char *buf, uint32_t pos, uint32_t max, uint16_t value)

  Append uint16_t as four hex characters to buffer.
- RX_STATIC_TESTABLE uint32_t internal_append_hex_u32 (char *buf, uint32_t pos, uint32_t max, uint32_t value)

  Append uint32_t as eight hex characters to buffer.
- RX_STATIC_TESTABLE uint32_t internal_append_decimal_u16 (char *buf, uint32_t pos, uint32_t max, uint16_t value)

  Append uint16_t as decimal digits to buffer.

- RX_STATIC_TESTABLE bool internal_is_printable (uint8_t c)

  Check if byte value is printable ASCII character.
- static uint32_t internal_format_hex_line (char *buf, uint32_t pos, uint32_t max, const uint8_t *payload, uint16_t offset, uint16_t line_bytes)

  Format payload as hex dump with ASCII sidebar.
- RX_STATIC_TESTABLE uint32_t internal_format_payload (char *buf, uint32_t pos, uint32_t max, const uint8_t *payload, uint16_t length)
- RX_STATIC_TESTABLE uint32_t internal_format_header (char *buf, uint32_t pos, uint32_t max, const rx_frame_header_t *header, bool is_tx)

  Format frame header metadata as single text line.
- const char * rx_frame_ascii_type_name (rx_frame_type_t type)

  Convert frame type enum to human-readable string.
- RX_STATIC_TESTABLE uint32_t internal_append_flag (char *buffer, uint32_t pos, uint32_t max, bool *first, uint8_t flags, uint8_t flag_bit, const char *flag_name)

  Append a single flag name with pipe separator if needed.
- const char * rx_frame_ascii_flags_str (uint8_t flags, char *buffer, uint32_t buffer_len)

  Convert frame flags bitmask to human-readable string.
- rx_err_t rx_frame_ascii_format (const rx_frame_t *frame, bool is_tx, char *output, uint32_t output_max_len, uint32_t *output_len)

  Format frame as human-readable ASCII string.

Variables

- static const char s_hex_chars [] = "0123456789ABCDEF"

  Lookup table for hexadecimal digit characters.

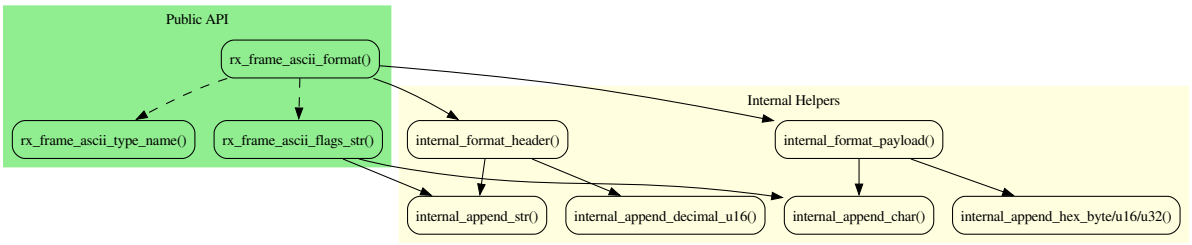
6.3.1 Detailed Description

Frame ASCII Formatter Implementation.

Implements the ASCII frame formatting functions declared in rx_frame_ascii.h. This module converts binary protocol frames into human-readable text for debugging and diagnostic output.

Implementation Overview

The implementation uses a functional decomposition approach with small, focused helper functions for each formatting task:



Design Decisions

- No snprintf/printf: Avoids libc dependencies, non-reentrant functions, and format string vulnerabilities. Custom formatters are safer and smaller.
- Position-based appending: All append helpers track position and bounds, preventing buffer overflows automatically.
- Static lookup table: Hex digits use compile-time lookup for speed.
- Consistent signature: All append helpers follow (buf, pos, max, value) convention for predictable composition.

Memory Safety

Every function validates bounds before writing. Output buffer overflow is impossible if functions are used correctly. Truncation is detected and reported via return value.

Performance Optimizations

- Hex digit lookup table (s_hex_chars) avoids arithmetic
- Direct character output avoids format string parsing
- Single-pass formatting (no temporary buffers except small decimal conversion)
- All loops have static upper bounds (NASA Rule 2)

NASA Power of 10 Compliance

- Rule 1: [OK] No goto, setjmp, or recursion
- Rule 2: [OK] All loops bounded (payload length, constant limits)
- Rule 3: [OK] No dynamic memory allocation
- Rule 4: [OK] Functions under 60 lines
- Rule 5: [OK] Input validation on all functions
- Rule 6: [OK] Variables at smallest scope
- Rule 7: [OK] Return values checked
- Rule 8: [OK] C23 typed enums for constants
- Rule 10: [OK] Compiles with -Wall -Wextra -Werror

See also

rx_frame_ascii.h Public API documentation

Author

Locked, Inc.

Date

2026-01-26

Copyright

Since

Version 1.0.0

Definition in file rx_frame_ascii.c.

6.3.2 Enumeration Type Documentation

6.3.2.1 buffer_size_constants_t

enum buffer_size_constants_t : uint8_t

Buffer size and offset constants for string building.

Constants for buffer boundary calculations and hex character sizing. All values fit in uint8_t for efficient comparison operations.

Since

Version 1.0.0

Enumerator

k_empty_buffer	Sentinel for empty/invalid buffer size. Value: 0
k_null_terminator	Space required for null terminator byte. Value: 1 byte
k_hex_chars_per_byte	Hex characters per byte (e.g., "FF"). Value: 2
k_hex_chars_per_u16	Hex characters for uint16_t (e.g., "FFFF"). Value: 4
k_hex_chars_per_u32	Hex characters for uint32_t (e.g., "FFFFFFFF"). Value: 8
k_u16_high_byte_shift	Bit shift to extract high byte from uint16_t. Value: 8 bits
k_u32_high_u16_shift	Bit shift to extract high uint16 from uint32. Value: 16 bits

Definition at line 177 of file rx_frame_ascii.c.

```
00177             : uint8_t {
00178   k_empty_buffer = 0, /**< Sentinel for empty/invalid buffer size.
00179                          @par Value: 0 */
00180
00181   k_null_terminator = 1, /**< Space required for null terminator byte.
00182                             @par Value: 1 byte */
00183
00184   k_hex_chars_per_byte = 2, /**< Hex characters per byte (e.g., "FF").
00185                                @par Value: 2 */
00186
00187   k_hex_chars_per_u16 = 4, /**< Hex characters for uint16_t (e.g., "FFFF").
00188                               @par Value: 4 */
00189
00190   k_hex_chars_per_u32 = 8, /**< Hex characters for uint32_t (e.g., "FFFFFFFF").
00191                               @par Value: 8 */
00192
00193   k_u16_high_byte_shift = 8, /**< Bit shift to extract high byte from uint16_t.
00194                                 @par Value: 8 bits */
00195
00196   k_u32_high_u16_shift = 16, /**< Bit shift to extract high uint16 from uint32.
00197                                 @par Value: 16 bits */
00198 } buffer_size_constants_t;
```

6.3.2.2 decimal_buf_constants_t

enum decimal_buf_constants_t : uint8_t

Decimal conversion buffer size constants.

Defines buffer sizes for temporary decimal string conversion.

Since

Version 1.0.0

Enumerator

k_u16_decimal_buf_size	Buffer size for uint16_t decimal with null terminator (6 bytes). Calculation: 5 digits + 1 null

Definition at line 209 of file rx_frame_ascii.c.

```
00209             : uint8_t {
00210   k_u16_decimal_buf_size = k_u16_decimal_max_digits + 1, /**< Buffer size for uint16_t decimal
00211                                                             with null terminator (6 bytes).
00212                                                             @par Calculation: 5 digits + 1 null */
00213 } decimal_buf_constants_t;
```

6.3.2.3 format_constants_t

enum format_constants_t : uint16_t

String formatting constants for hex and decimal conversion.

Internal constants used by the formatting helper functions. These define bit manipulation parameters, ASCII character ranges, and output formatting dimensions.

Hex Digit Extraction

To convert a byte to two hex characters:

- Upper nibble: (byte >> k_hex_digit_shift) & k_hex_digit_mask
- Lower nibble: byte & k_hex_digit_mask

Printable ASCII Range

Characters 0x20 (space) through 0x7E (~) are printable. Outside this range, the hex dump ASCII sidebar displays '.' as a placeholder.

Since

Version 1.0.0

Enumerator

k_hex_digit_shift	Bits to shift for upper hex nibble extraction. Value: 4 (half a byte = 4 bits)
k_hex_digit_mask	Mask for extracting single hex digit (4 bits). Value: 0x0F (binary: 00001111)
k_printable_min	Minimum printable ASCII character (space). Value: 0x20 (32 decimal) See also internal_is_printable()
k_printable_max	Maximum printable ASCII character (tilde). Value: 0x7E (126 decimal) See also internal_is_printable()
k_bytes_per_line	Bytes per line in hex dump output. Matches standard xxd/hexdump format. Value: 16 bytes

k_min_output_size	Minimum output buffer size accepted. Fits header + CRC with empty payload. Value: 64 bytes
k_hex_space_gap	Spaces between hex and ASCII sections. Value: 2 spaces
k_mask_u8	Mask for uint8_t extraction from larger type. Value: 0xFF (8 bits)
k_mask_u16	Mask for uint16_t extraction from uint32_t. Value: 0xFFFF (16 bits)
k_decimal_base	Decimal number base for digit extraction. Value: 10
k_u16_decimal_max_digits	Maximum decimal digits for uint16_t (65535). Value: 5
k_flags_buf_size	Buffer size for flags string formatting. Fits "ACK|RETX|PRI|FEC|SOFT" + null. Value: 32 bytes

Definition at line 124 of file rx_frame_ascii.c.

```
00124           : uint16_t {
00125     k_hex_digit_shift = 4, /**< Bits to shift for upper hex nibble extraction.
00126                            @par Value: 4 (half a byte = 4 bits) */
00127
00128     k_hex_digit_mask = 0xF, /**< Mask for extracting single hex digit (4 bits).
00129                             @par Value: 0x0F (binary: 00001111) */
00130
00131     k_printable_min = 0x20, /**< Minimum printable ASCII character (space).
00132                             @par Value: 0x20 (32 decimal)
00133                             @see internal_is_printable() */
00134
00135     k_printable_max = 0x7E, /**< Maximum printable ASCII character (tilde).
00136                             @par Value: 0x7E (126 decimal)
00137                             @see internal_is_printable() */
00138
00139     k_bytes_per_line = 16, /**< Bytes per line in hex dump output.
00140                            Matches standard xxd/hexdump format.
00141                            @par Value: 16 bytes */
00142
00143     k_min_output_size = 64, /**< Minimum output buffer size accepted.
00144                             Fits header + CRC with empty payload.
00145                             @par Value: 64 bytes */
```

```
00146
00147   k_hex_space_gap = 2, /**< Spaces between hex and ASCII sections.
00148                         @par Value: 2 spaces */
00149
00150   k_mask_u8 = 0xFF, /**< Mask for uint8_t extraction from larger type.
00151                      @par Value: 0xFF (8 bits) */
00152
00153   k_mask_u16 = 0xFFFF, /**< Mask for uint16_t extraction from uint32_t.
00154                         @par Value: 0xFFFF (16 bits) */
00155
00156   k_decimal_base = 10, /**< Decimal number base for digit extraction.
00157                         @par Value: 10 */
00158
00159   k_u16_decimal_max_digits = 5, /**< Maximum decimal digits for uint16_t (65535).
00160                                  @par Value: 5 */
00161
00162   k_flags_buf_size = 32, /**< Buffer size for flags string formatting.
00163                           Fits "ACK|RETX|PRI|FEC|SOFT" + null.
00164                           @par Value: 32 bytes */
00165 } format_constants_t;
```

6.3.3 Function Documentation

6.3.3.1 internal_format_payload()

```
RX_STATIC_TESTABLE uint32_t internal_format_payload (
                char * buf,
                uint32_t pos,
                uint32_t max,
                const uint8_t * payload,
                uint16_t length)
```

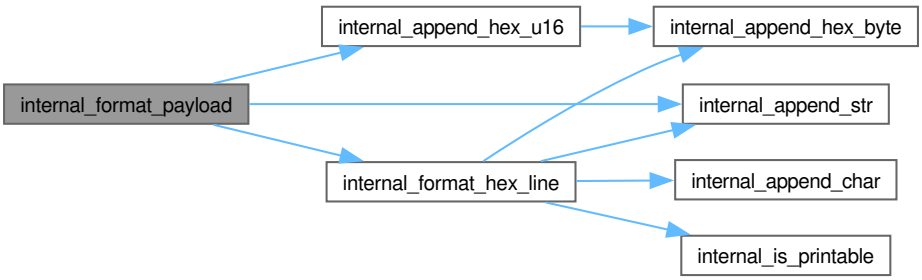
Definition at line 618 of file rx_frame_ascii.c.

```
00623 {
00624   /* Rule 5: Pre-condition invariant - callers guarantee these conditions */
00625   RX_ASSERT(buf != nullptr && max != k_empty_buffer && pos < max,
00626             "internal_format_payload: precondition violated by caller");
00627
00628   if (length == 0) {
00629     pos = internal_append_str(buf, pos, max, "[PAYLOAD] (empty)\r\n");
00630     return pos;
00631   }
00632
00633   /* payload must be non-NULL when length > 0 */
00634   if (payload == nullptr) {
00635     return pos;
00636   }
00637
00638   uint16_t offset = 0;
00639   while (offset < length) {
00640     /* Start new line with offset */
00641     pos = internal_append_str(buf, pos, max, "[");
00642     pos = internal_append_hex_u16(buf, pos, max, offset);
00643     pos = internal_append_str(buf, pos, max, "] ");
00644
00645     /* Calculate bytes on this line */
00646     uint16_t line_bytes = length - offset;
00647     if (line_bytes > k_bytes_per_line) {
00648       line_bytes = k_bytes_per_line;
00649     }
00650
00651     /* Format hex bytes and ASCII sidebar */
00652     pos = internal_format_hex_line(buf, pos, max, payload, offset, line_bytes);
00653
00654     pos = internal_append_str(buf, pos, max, "\r\n");
00655     offset += line_bytes;
00656   }
00657
00658   return pos;
00659 }
```

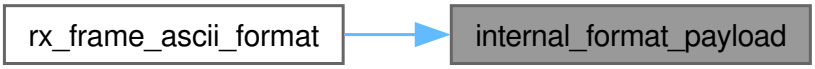
References internal_append_hex_u16(), internal_append_str(), internal_format_hex_line(), k_bytes_per_line, and k_empty_buffer.

Referenced by rx_frame_ascii_format().

Here is the call graph for this function:



Here is the caller graph for this function:



6.3.3.2 rx_frame_ascii_flags_str()

```
const char * rx_frame_ascii_flags_str (
            uint8_t flags,
            char * buffer,
            uint32_t buffer_len)
```

Convert frame flags bitmask to human-readable string.

Get frame flags as human-readable string.

Implementation of rx_frame_ascii_flags_str(). Iterates through known flag bits, appending names with pipe separators.

Note

Flags formatted as pipe-separated: "ACK|RETX|PRI"

See also

rx_frame_ascii.h for full documentation

Definition at line 840 of file rx_frame_ascii.c.

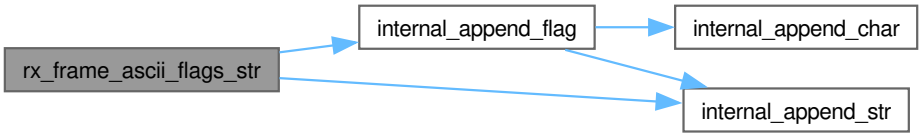
```
00841 {
00842   if (buffer == nullptr || buffer_len == k_empty_buffer) {
00843     return "";
00844   }
00845
00846   uint32_t pos = 0;
00847
00848   if (flags == k_frame_flag_none) {
00849     pos         = internal_append_str(buffer, pos, buffer_len, "NONE");
00850     buffer[pos] = '\0';
00851     return buffer;
00852   }
00853
00854   bool first = true;
00855
00856   /* Use helper to append each flag */
00857   pos =
00858     internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_requires_ack, "ACK");
00859   pos =
00860     internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_retransmit, "RETX");
00861   pos = internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_priority, "PRI");
```

```
00862   pos =
00863     internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_fec_enabled, "FEC");
00864   pos =
00865     internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_soft_nack, "SOFT");
00866
00867   buffer[pos] = '\0';
00868   return buffer;
00869 }
```

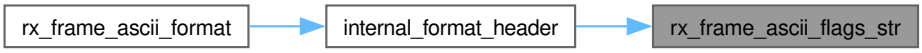
References internal_append_flag(), internal_append_str(), and k_empty_buffer.

Referenced by internal_format_header().

Here is the call graph for this function:



Here is the caller graph for this function:



6.3.3.3 rx_frame_ascii_format()

```
rx_err_t rx_frame_ascii_format (
                const rx_frame_t * frame,
                bool is_tx,
                char * output,
                uint32_t output_max_len,
                uint32_t * output_len)   [nodiscard]
```

Format frame as human-readable ASCII string.

Format binary frame as human-readable ASCII.

Implementation of rx_frame_ascii_format(). Orchestrates header, payload, and \rC formatting using internal helper functions.

Implementation Notes

- Validates all inputs before any output
- Calls internal_format_header() for metadata line
- Calls internal_format_payload() for hex dump
- Appends \rC footer directly
- Detects truncation and returns k_rx_err_no_mem

See also

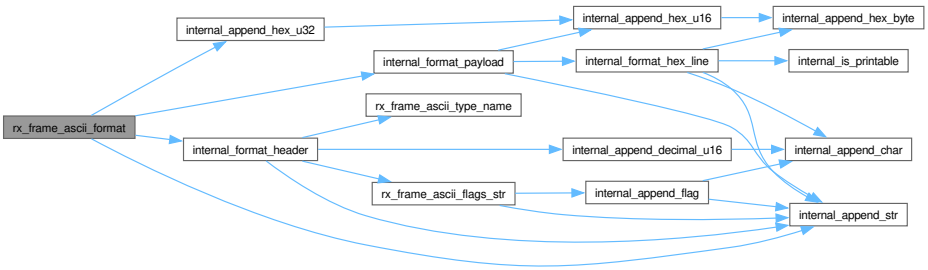
rx_frame_ascii.h for full documentation

Definition at line 889 of file rx_frame_ascii.c.

```
00894 {
00895   /* Rule 5: Pre-condition validation */
00896   if (frame == nullptr) {
00897     return k_rx_err_invalid_arg;
00898   }
00899   if (output == nullptr) {
00900     return k_rx_err_invalid_arg;
00901   }
00902   if (output_len == nullptr) {
00903     return k_rx_err_invalid_arg;
00904   }
00905   if (output_max_len < k_min_output_size) {
00906     return k_rx_err_no_mem;
00907   }
00908
00909   /* Validate payload length does not exceed frame capacity */
00910   if (frame->header.length > k_frame_max_payload) {
00911     return k_rx_err_invalid_arg;
00912   }
00913
00914   uint32_t pos = 0;
00915
00916   /* Format header metadata */
00917   pos = internal_format_header(output, pos, output_max_len, &frame->header, is_tx);
00918
00919   /* Payload hex dump - use validated length to prevent out-of-bounds reads */
00920   pos = internal_format_payload(output, pos, output_max_len, frame->payload, frame->header.length);
00921
00922   /* CRC */
00923   pos = internal_append_str(output, pos, output_max_len, "[CRC] ");
00924   pos = internal_append_hex_u32(output, pos, output_max_len, frame->crc);
00925   pos = internal_append_str(output, pos, output_max_len, "\r\n");
00926
00927   /* Rule 5: Post-condition: helpers cap pos at output_max_len - 1 via bounds
00928    * checks, so pos < output_max_len is an invariant by construction. */
00929
00930   /* Null terminate */
00931   output[pos] = '\0';
00932
00933   *output_len = pos;
00934
00935   return k_rx_ok;
00936 }
```

References internal_append_hex_u32(), internal_append_str(), internal_format_header(), internal_format_payload(), and k_min_output_size.

Here is the call graph for this function:



6.3.3.4 rx_frame_ascii_type_name()

```
const char * rx_frame_ascii_type_name (
            rx_frame_type_t type)
```

Convert frame type enum to human-readable string.

Get frame type as human-readable string.

Implementation of rx_frame_ascii_type_name(). Uses switch statement for O(1) lookup with compile-time string literals.

See also

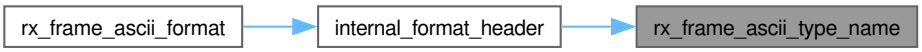
rx_frame_ascii.h for full documentation

Definition at line 758 of file rx_frame_ascii.c.

```
00759 {
00760   switch (type) {
00761     case k_frame_type_ping:
00762       return "PING";
00763     case k_frame_type_pong:
00764       return "PONG";
00765     case k_frame_type_command:
00766       return "COMMAND";
00767     case k_frame_type_response:
00768       return "RESPONSE";
00769     case k_frame_type_ack:
00770       return "ACK";
00771     case k_frame_type_nack:
00772       return "NACK";
00773     case k_frame_type_log_message:
00774       return "LOG_MESSAGE";
00775     case k_frame_type_reset_ack:
00776       return "RESET_ACK";
00777     case k_frame_type_reset:
00778       return "RESET";
00779     default:
00780       return "UNKNOWN";
00781   }
00782 }
```

Referenced by internal_format_header().

Here is the caller graph for this function:



6.3.4 Variable Documentation

6.3.4.1 s_hex_chars

const char s_hex_chars[] = "0123456789ABCDEF" [static]

Lookup table for hexadecimal digit characters.

Static constant lookup table mapping nibble values (0-15) to their ASCII hexadecimal character representation ('0'-'9', 'A'-'F'). Uses uppercase hex letters for consistency with standard debugger output.

Usage

```
uint8_t nibble = (byte » 4) & 0x0F;  // Extract upper nibble
char hex_char = s_hex_chars[nibble];  // Convert to hex character
```

Memory

17 bytes (16 characters + null terminator) in .rodata section

Note

Thread-safe: read-only constant data

Since

Version 1.0.0

Definition at line 237 of file rx_frame_ascii.c.

Referenced by internal_append_hex_byte().

6.4 rx˙frame˙ascii.c

Go to the documentation of this file.

```
00001 /**
00002  * @file rx_frame_ascii.c
00003  * @brief Frame ASCII Formatter Implementation
00004  *
00005  * @details
00006  * Implements the ASCII frame formatting functions declared in rx_frame_ascii.h.
00007  * This module converts binary protocol frames into human-readable text for
00008  * debugging and diagnostic output.
00009  *
00010  * @par Implementation Overview
00011  * The implementation uses a functional decomposition approach with small,
00012  * focused helper functions for each formatting task:
00013  *
00014  * @dot
00015  * digraph impl_structure {
00016  *   rankdir=TB;
00017  *   node [shape=box, style=rounded];
00018  *
00019  *   subgraph cluster_public {
00020  *     label="Public API";
00021  *     style=filled;
00022  *     color=lightgreen;
00023  *     format [label="rx_frame_ascii_format()"];
00024  *     type_name [label="rx_frame_ascii_type_name()"];
00025  *     flags_str [label="rx_frame_ascii_flags_str()"];
00026  *   }
00027  *
00028  *   subgraph cluster_helpers {
00029  *     label="Internal Helpers";
00030  *     style=filled;
00031  *     color=lightyellow;
00032  *     append_str [label="internal_append_str()"];
00033  *     append_char [label="internal_append_char()"];
00034  *     append_hex [label="internal_append_hex_byte/u16/u32()"];
00035  *     append_dec [label="internal_append_decimal_u16()"];
00036  *     format_header [label="internal_format_header()"];
00037  *     format_payload [label="internal_format_payload()"];
00038  *   }
00039  *
00040  *   format -> format_header;
00041  *   format -> format_payload;
00042  *   format -> type_name [style=dashed];
00043  *   format -> flags_str [style=dashed];
00044  *   format_header -> append_str;
00045  *   format_header -> append_dec;
00046  *   format_payload -> append_hex;
00047  *   format_payload -> append_char;
00048  *   flags_str -> append_str;
00049  *   flags_str -> append_char;
00050  * }
00051  * @enddot
00052  *
00053  * @par Design Decisions
00054  * - **No snprintf/printf**: Avoids libc dependencies, non-reentrant functions,
00055  *   and format string vulnerabilities. Custom formatters are safer and smaller.
00056  * - **Position-based appending**: All append helpers track position and bounds,
00057  *   preventing buffer overflows automatically.
00058  * - **Static lookup table**: Hex digits use compile-time lookup for speed.
00059  * - **Consistent signature**: All append helpers follow (buf, pos, max, value)
00060  *   convention for predictable composition.
00061  *
00062  * @par Memory Safety
00063  * Every function validates bounds before writing. Output buffer overflow is
00064  * impossible if functions are used correctly. Truncation is detected and
00065  * reported via return value.
00066  *
00067  * @par Performance Optimizations
00068  * - Hex digit lookup table (s_hex_chars) avoids arithmetic
00069  * - Direct character output avoids format string parsing
00070  * - Single-pass formatting (no temporary buffers except small decimal conversion)
00071  * - All loops have static upper bounds (NASA Rule 2)
00072  *
00073  * @par NASA Power of 10 Compliance
00074  * - Rule 1: [OK] No goto, setjmp, or recursion
00075  * - Rule 2: [OK] All loops bounded (payload length, constant limits)
00076  * - Rule 3: [OK] No dynamic memory allocation
00077  * - Rule 4: [OK] Functions under 60 lines
00078  * - Rule 5: [OK] Input validation on all functions
00079  * - Rule 6: [OK] Variables at smallest scope
00080  * - Rule 7: [OK] Return values checked
00081  * - Rule 8: [OK] C23 typed enums for constants
00082  * - Rule 10: [OK] Compiles with -Wall -Wextra -Werror
```

```
00083  *
00084  * @see rx_frame_ascii.h Public API documentation
00085  *
00086  * @author Locked, Inc.
00087  * @date 2026-01-26
00088  * @copyright Copyright (c) 2026 Locked Inc.
00089  * SPDX-License-Identifier: MIT
00090  * @since Version 1.0.0
00091  */
00092
00093 #include "rx_frame_ascii.h"
00094
00095 #include <string.h>
00096
00097 #include "rx_check.h"
00098
00099 /*
       ================================================================================
00100  * Private Constants
00101  *
       ================================================================================
00102  */
00103
00104 /**
00105  * @enum format_constants_t
00106  * @brief String formatting constants for hex and decimal conversion
00107  *
00108  * @details
00109  * Internal constants used by the formatting helper functions. These define
00110  * bit manipulation parameters, ASCII character ranges, and output formatting
00111  * dimensions.
00112  *
00113  * @par Hex Digit Extraction
00114  * To convert a byte to two hex characters:
00115  * - Upper nibble: `(byte » k_hex_digit_shift) & k_hex_digit_mask‘
00116  * - Lower nibble: `byte & k_hex_digit_mask‘
00117  *
00118  * @par Printable ASCII Range
00119  * Characters 0x20 (space) through 0x7E (~) are printable. Outside this range,
00120  * the hex dump ASCII sidebar displays '.' as a placeholder.
00121  *
00122  * @since Version 1.0.0
00123  */
00124 typedef enum : uint16_t {
00125   k_hex_digit_shift = 4, /**< Bits to shift for upper hex nibble extraction.
00126                             @par Value: 4 (half a byte = 4 bits) */
00127
00128   k_hex_digit_mask = 0xF, /**< Mask for extracting single hex digit (4 bits).
00129                              @par Value: 0x0F (binary: 00001111) */
00130
00131   k_printable_min = 0x20, /**< Minimum printable ASCII character (space).
00132                              @par Value: 0x20 (32 decimal)
00133                              @see internal_is_printable() */
00134
00135   k_printable_max = 0x7E, /**< Maximum printable ASCII character (tilde).
00136                              @par Value: 0x7E (126 decimal)
00137                              @see internal_is_printable() */
00138
00139   k_bytes_per_line = 16, /**< Bytes per line in hex dump output.
00140                             Matches standard xxd/hexdump format.
00141                             @par Value: 16 bytes */
00142
00143   k_min_output_size = 64, /**< Minimum output buffer size accepted.
00144                              Fits header + CRC with empty payload.
00145                              @par Value: 64 bytes */
00146
00147   k_hex_space_gap = 2, /**< Spaces between hex and ASCII sections.
00148                           @par Value: 2 spaces */
00149
00150   k_mask_u8 = 0xFF, /**< Mask for uint8_t extraction from larger type.
00151                        @par Value: 0xFF (8 bits) */
00152
00153   k_mask_u16 = 0xFFFF, /**< Mask for uint16_t extraction from uint32_t.
00154                           @par Value: 0xFFFF (16 bits) */
00155
00156   k_decimal_base = 10, /**< Decimal number base for digit extraction.
00157                           @par Value: 10 */
00158
00159   k_u16_decimal_max_digits = 5, /**< Maximum decimal digits for uint16_t (65535).
00160                                    @par Value: 5 */
00161
00162   k_flags_buf_size = 32, /**< Buffer size for flags string formatting.
00163                             Fits "ACK|RETX|PRI|FEC|SOFT" + null.
00164                             @par Value: 32 bytes */
00165 } format_constants_t;
00166
00167 /**
```

```
00168  * @enum buffer_size_constants_t
00169  * @brief Buffer size and offset constants for string building
00170  *
00171  * @details
00172  * Constants for buffer boundary calculations and hex character sizing.
00173  * All values fit in uint8_t for efficient comparison operations.
00174  *
00175  * @since Version 1.0.0
00176  */
00177 typedef enum : uint8_t {
00178     k_empty_buffer = 0, /**< Sentinel for empty/invalid buffer size.
00179                          @par Value: 0 */
00180
00181     k_null_terminator = 1, /**< Space required for null terminator byte.
00182                             @par Value: 1 byte */
00183
00184     k_hex_chars_per_byte = 2, /**< Hex characters per byte (e.g., "FF").
00185                                @par Value: 2 */
00186
00187     k_hex_chars_per_u16 = 4, /**< Hex characters for uint16_t (e.g., "FFFF").
00188                               @par Value: 4 */
00189
00190     k_hex_chars_per_u32 = 8, /**< Hex characters for uint32_t (e.g., "FFFFFFFF").
00191                               @par Value: 8 */
00192
00193     k_u16_high_byte_shift = 8, /**< Bit shift to extract high byte from uint16_t.
00194                                 @par Value: 8 bits */
00195
00196     k_u32_high_u16_shift = 16, /**< Bit shift to extract high uint16 from uint32.
00197                                 @par Value: 16 bits */
00198 } buffer_size_constants_t;
00199
00200 /**
00201  * @enum decimal_buf_constants_t
00202  * @brief Decimal conversion buffer size constants
00203  *
00204  * @details
00205  * Defines buffer sizes for temporary decimal string conversion.
00206  *
00207  * @since Version 1.0.0
00208  */
00209 typedef enum : uint8_t {
00210     k_u16_decimal_buf_size = k_u16_decimal_max_digits + 1, /**< Buffer size for uint16_t decimal
00211                                                             with null terminator (6 bytes).
00212                                                             @par Calculation: 5 digits + 1 null */
00213 } decimal_buf_constants_t;
00214
00215 /**
00216  * @var s_hex_chars
00217  * @brief Lookup table for hexadecimal digit characters
00218  *
00219  * @details
00220  * Static constant lookup table mapping nibble values (0-15) to their ASCII
00221  * hexadecimal character representation ('0'-'9', 'A'-'F'). Uses uppercase
00222  * hex letters for consistency with standard debugger output.
00223  *
00224  * @par Usage
00225  * @code
00226  * uint8_t nibble = (byte » 4) & 0x0F;  // Extract upper nibble
00227  * char hex_char = s_hex_chars[nibble];   // Convert to hex character
00228  * @endcode
00229  *
00230  * @par Memory
00231  * 17 bytes (16 characters + null terminator) in .rodata section
00232  *
00233  * @note Thread-safe: read-only constant data
00234  *
00235  * @since Version 1.0.0
00236  */
00237 static const char s_hex_chars[] = "0123456789ABCDEF";
00238
00239 /*
      ============================================================================
00240  * Private Helper Functions
00241  *
      ============================================================================
00242  */
00243
00244 /**
00245  * @defgroup frame_ascii_helpers Internal Helper Functions
00246  * @{
00247  *
00248  * @brief String building helper functions with bounds checking
00249  *
00250  * @details
00251  * All append helpers follow a consistent signature pattern for composability:
00252  * ```c
```

```
00253  * uint32_t internal_append_xxx(char* buf, uint32_t pos, uint32_t max, <value>);
00254  * ```
00255  *
00256  * @par Parameters
00257  * - **buf**: Output buffer pointer (may be NULL for safety)
00258  * - **pos**: Current write position in buffer
00259  * - **max**: Maximum buffer size (total capacity)
00260  * - **value**: Data to append (varies by function)
00261  *
00262  * @par Return Value
00263  * All helpers return the updated position after appending. If the append
00264  * would overflow, the original position is returned unchanged.
00265  *
00266  * @par Bounds Safety
00267  * - All functions check `pos < max - k_null_terminator` before writing
00268  * - NULL buffer pointer causes immediate return with unchanged position
00269  * - Zero-size buffer causes immediate return with unchanged position
00270  * - No writes beyond `max - 1` to preserve space for null terminator
00271  *
00272  * @par Usage Pattern
00273  * ```c
00274  * uint32_t pos = 0;
00275  * pos = internal_append_str(buf, pos, max, "Hello ");
00276  * pos = internal_append_decimal_u16(buf, pos, max, 42);
00277  * pos = internal_append_char(buf, pos, max, '!');
00278  * buf[pos] = '\0';  // Null terminate
00279  * // Result: "Hello 42!"
00280  * ```
00281  * @}
00282  */
00283
00284 /**
00285  * @brief Append string to buffer with bounds checking
00286  *
00287  * @details
00288  * Copies characters from source string to output buffer until null terminator
00289  * is reached or buffer is full (leaving room for final null terminator).
00290  *
00291  *
00292  *
00293  * @pre pos < max (for any writing to occur)
00294  *
00295  * @post Characters written in range [pos, return_value)
00296  * @post Buffer NOT null-terminated (caller responsibility)
00297  *
00298  * @note Does not write null terminator
00299  * @note Returns original pos if nothing can be written
00300  *
00301  * @ingroup frame_ascii_helpers
00302  * @since Version 1.0.0
00303  */
00304 RX_STATIC_TESTABLE uint32_t internal_append_str(char*       buf,
00305                                                 uint32_t    pos,
00306                                                 uint32_t    max,
00307                                                 const char* str)
00308 {
00309   /* Rule 5: Pre-condition invariant - callers guarantee these conditions */
00310   RX_ASSERT(buf != nullptr && str != nullptr && max != k_empty_buffer && pos < max,
00311             "internal_append_str: precondition violated by caller");
00312
00313   while (pos < max - k_null_terminator && *str != '\0') {
00314     buf[pos++] = *str++;
00315   }
00316   return pos;
00317 }
00318
00319 /**
00320  * @brief Append single character to buffer with bounds checking
00321  *
00322  * @details
00323  * Writes a single character to the buffer at the current position if space
00324  * is available (accounting for null terminator).
00325  *
00326  *
00327  *
00328  * @pre pos < max - 1 (for writing to occur)
00329  *
00330  * @post buf[pos] contains c (if written)
00331  *
00332  * @ingroup frame_ascii_helpers
00333  * @since Version 1.0.0
00334  */
00335 RX_STATIC_TESTABLE uint32_t internal_append_char(char* buf, uint32_t pos, uint32_t max, char c)
00336 {
00337   /* Rule 5: Pre-condition invariant - callers guarantee these conditions */
00338   RX_ASSERT(buf != nullptr && max != k_empty_buffer && pos < max,
00339             "internal_append_char: precondition violated by caller");
```

```
00340
00341   if (pos < max - k_null_terminator) {
00342     buf[pos++] = c;
00343   }
00344   return pos;
00345 }
00346
00347 /**
00348  * @brief Append byte as two hex characters to buffer
00349  *
00350  * @details
00351  * Converts a single byte to its two-character hexadecimal representation
00352  * (e.g., 0xFF -> "FF") and appends it to the buffer. Uses uppercase hex
00353  * letters via s_hex_chars lookup table.
00354  *
00355  *
00356  *
00357  * @pre pos + 2 <= max - 1 (for writing to occur)
00358  *
00359  * @post buf[pos] contains upper nibble hex char
00360  * @post buf[pos+1] contains lower nibble hex char
00361  *
00362  * @par Example Output
00363  * - byte=0x00 -> "00"
00364  * - byte=0xFF -> "FF"
00365  * - byte=0x5A -> "5A"
00366  *
00367  * @ingroup frame_ascii_helpers
00368  * @since Version 1.0.0
00369  */
00370 RX_STATIC_TESTABLE uint32_t internal_append_hex_byte(char*    buf,
00371                                                      uint32_t pos,
00372                                                      uint32_t max,
00373                                                      uint8_t  byte)
00374 {
00375   /* Rule 5: Pre-condition invariant - callers guarantee these conditions */
00376   RX_ASSERT(buf != nullptr && max >= k_hex_chars_per_byte && pos < max,
00377             "internal_append_hex_byte: precondition violated by caller");
00378
00379   if (pos < max - k_hex_chars_per_byte) {
00380     buf[pos++] = s_hex_chars[(byte » k_hex_digit_shift) & k_hex_digit_mask];
00381     buf[pos++] = s_hex_chars[byte & k_hex_digit_mask];
00382   }
00383   return pos;
00384 }
00385
00386 /**
00387  * @brief Append uint16_t as four hex characters to buffer
00388  *
00389  * @details
00390  * Converts a 16-bit value to its four-character hexadecimal representation
00391  * (e.g., 0xABCD -> "ABCD") in big-endian order (most significant byte first).
00392  *
00393  *
00394  *
00395  * @pre pos + 4 <= max - 1 (for writing to occur)
00396  *
00397  * @post Four hex characters written at buf[pos..pos+3]
00398  *
00399  * @par Example Output
00400  * - value=0x0000 -> "0000"
00401  * - value=0xFFFF -> "FFFF"
00402  * - value=0x1234 -> "1234"
00403  *
00404  * @ingroup frame_ascii_helpers
00405  * @since Version 1.0.0
00406  */
00407 RX_STATIC_TESTABLE uint32_t internal_append_hex_u16(char*    buf,
00408                                                     uint32_t pos,
00409                                                     uint32_t max,
00410                                                     uint16_t value)
00411 {
00412   /* Rule 5: Pre-condition invariant - callers guarantee these conditions */
00413   RX_ASSERT(buf != nullptr && max >= k_hex_chars_per_u16 && pos < max,
00414             "internal_append_hex_u16: precondition violated by caller");
00415
00416   pos = internal_append_hex_byte(buf, pos, max, (uint8_t)(value » k_u16_high_byte_shift));
00417   pos = internal_append_hex_byte(buf, pos, max, (uint8_t)(value & k_mask_u8));
00418   return pos;
00419 }
00420
00421 /**
00422  * @brief Append uint32_t as eight hex characters to buffer
00423  *
00424  * @details
00425  * Converts a 32-bit value to its eight-character hexadecimal representation
00426  * (e.g., 0xDEADBEEF -> "DEADBEEF") in big-endian order. Commonly used for
```

```
00427  * \\rC-32 values in frame output.
00428  *
00429  *
00430  *
00431  * @pre pos + 8 <= max - 1 (for writing to occur)
00432  *
00433  * @post Eight hex characters written at buf[pos..pos+7]
00434  *
00435  * @par Example Output
00436  * - value=0x00000000 -> "00000000"
00437  * - value=0xFFFFFFFF -> "FFFFFFFF"
00438  * - value=0xDEADBEEF -> "DEADBEEF"
00439  *
00440  * @ingroup frame_ascii_helpers
00441  * @since Version 1.0.0
00442  */
00443 RX_STATIC_TESTABLE uint32_t internal_append_hex_u32(char*    buf,
00444                                                     uint32_t pos,
00445                                                     uint32_t max,
00446                                                     uint32_t value)
00447 {
00448   /* Rule 5: Pre-condition invariant - callers guarantee these conditions */
00449   RX_ASSERT(buf != nullptr && max >= k_hex_chars_per_u32 && pos < max,
00450             "internal_append_hex_u32: precondition violated by caller");
00451
00452   pos = internal_append_hex_u16(buf, pos, max, (uint16_t)(value » k_u32_high_u16_shift));
00453   pos = internal_append_hex_u16(buf, pos, max, (uint16_t)(value & k_mask_u16));
00454   return pos;
00455 }
00456
00457 /**
00458  * @brief Append uint16_t as decimal digits to buffer
00459  *
00460  * @details
00461  * Converts a 16-bit value to its decimal string representation (1-5 digits).
00462  * Uses a small stack buffer for digit reversal, avoiding heap allocation.
00463  * Zero value outputs single '0' character.
00464  *
00465  * @par Algorithm
00466  * 1. Extract digits via repeated modulo/division (reverse order)
00467  * 2. Store in temporary stack buffer
00468  * 3. Reverse and append to output buffer
00469  *
00470  *
00471  *
00472  * @pre Sufficient space for up to 5 digits
00473  *
00474  * @post Decimal digits written without leading zeros (except for value=0)
00475  *
00476  * @par Example Output
00477  * - value=0 -> "0"
00478  * - value=1 -> "1"
00479  * - value=12345 -> "12345"
00480  * - value=65535 -> "65535"
00481  *
00482  * @par Stack Usage
00483  * ~6 bytes (temp buffer for digit reversal)
00484  *
00485  * @ingroup frame_ascii_helpers
00486  * @since Version 1.0.0
00487  */
00488 RX_STATIC_TESTABLE uint32_t internal_append_decimal_u16(char*    buf,
00489                                                         uint32_t pos,
00490                                                         uint32_t max,
00491                                                         uint16_t value)
00492 {
00493   /* Rule 5: Pre-condition invariant - callers guarantee these conditions */
00494   RX_ASSERT(buf != nullptr && max != k_empty_buffer && pos < max,
00495             "internal_append_decimal_u16: precondition violated by caller");
00496
00497   char    temp[k_u16_decimal_buf_size]; /* Max 5 digits + null for uint16_t */
00498   uint8_t len = 0;
00499
00500   if (value == 0) {
00501     return internal_append_char(buf, pos, max, '0');
00502   }
00503
00504   while (value > 0) {
00505     const uint8_t digit = (uint8_t)(value % k_decimal_base);
00506     temp[len++]         = (char)('0' + digit);
00507     value /= k_decimal_base;
00508   }
00509
00510   /* Reverse digits into output */
00511   while (len > 0) {
00512     pos = internal_append_char(buf, pos, max, temp[--len]);
00513   }
```

```
00514
00515   return pos;
00516 }
00517
00518 /**
00519  * @brief Check if byte value is printable ASCII character
00520  *
00521  * @details
00522  * Tests whether a byte value falls within the printable ASCII range
00523  * (0x20-0x7E inclusive). Used to decide whether to display the actual
00524  * character or a '.' placeholder in hex dump ASCII sidebars.
00525  *
00526  * @par Printable Range
00527  * - 0x20 (space) through 0x7E (~) are printable
00528  * - 0x00-0x1F are control characters (not printable)
00529  * - 0x7F (DEL) is not printable
00530  * - 0x80-0xFF are extended ASCII (not printable in this implementation)
00531  *
00532  *
00533  *
00534  * @note Pure function with no side effects
00535  *
00536  * @ingroup frame_ascii_helpers
00537  * @since Version 1.0.0
00538  */
00539 RX_STATIC_TESTABLE bool internal_is_printable(uint8_t c)
00540 {
00541   return (bool)((c >= k_printable_min) && (c <= k_printable_max));
00542 }
00543
00544 /**
00545  * @brief Format payload as hex dump with ASCII sidebar
00546  *
00547  * @details
00548  * Formats binary payload data as a traditional hex dump with 16 bytes per line,
00549  * offset prefix, and ASCII sidebar showing printable characters (or '.' for
00550  * non-printable bytes).
00551  *
00552  * @par Output Format
00553  * Each line follows this structure:
00554  * ```
00555  * [NNNN] XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  ................
00556  * ```
00557  * Where:
00558  * - `[NNNN]` is the 4-digit hex offset from payload start
00559  * - `XX XX ...` is the hex representation of up to 16 bytes
00560  * - ASCII sidebar shows printable chars or '.' for non-printable
00561  *
00562  * @par Empty Payload Handling
00563  * If length is 0, outputs: `[PAYLOAD] (empty)\\r\\n`
00564  *
00565  *
00566  *
00567  * @pre buf != nullptr (for any output)
00568  * @pre payload != nullptr if length > 0
00569  *
00570  * @post Hex dump lines written with \\r\\n line endings
00571  *
00572  * @par Loop Bounds (NASA Rule 2)
00573  * - Outer loop: bounded by `length / k_bytes_per_line + 1` iterations
00574  * - Inner loops: bounded by `k_bytes_per_line` (16) iterations
00575  *
00576  * @ingroup frame_ascii_helpers
00577  * @since Version 1.0.0
00578  */
00579 /**
00580  * @brief Format one hex dump line (hex bytes + ASCII sidebar)
00581  *
00582  *
00583  *
00584  * @ingroup frame_ascii_helpers
00585  * @since Version 1.0.0
00586  */
00587 static uint32_t internal_format_hex_line(char*          buf,
00588                                          uint32_t       pos,
00589                                          uint32_t       max,
00590                                          const uint8_t* payload,
00591                                          uint16_t       offset,
00592                                          uint16_t       line_bytes)
00593 {
00594   /* Hex dump */
00595   for (uint16_t i = 0; i < k_bytes_per_line; i++) {
00596     if (i < line_bytes) {
00597       pos = internal_append_hex_byte(buf, pos, max, payload[offset + i]);
00598       pos = internal_append_char(buf, pos, max, ' ');
00599     } else {
00600       pos = internal_append_str(buf, pos, max, "   "); /* Pad short lines */
```

```
00601     }
00602   }
00603
00604   /* ASCII sidebar */
00605   pos = internal_append_char(buf, pos, max, ' ');
00606   for (uint16_t i = 0; i < line_bytes; i++) {
00607     const uint8_t c          = payload[offset + i];
00608     char          display_ch = '.';
00609     if (internal_is_printable(c)) {
00610       display_ch = (char)c;
00611     }
00612     pos = internal_append_char(buf, pos, max, display_ch);
00613   }
00614
00615   return pos;
00616 }
00617
00618 RX_STATIC_TESTABLE uint32_t internal_format_payload(char*          buf,
00619                                                     uint32_t       pos,
00620                                                     uint32_t       max,
00621                                                     const uint8_t* payload,
00622                                                     uint16_t       length)
00623 {
00624   /* Rule 5: Pre-condition invariant - callers guarantee these conditions */
00625   RX_ASSERT(buf != nullptr && max != k_empty_buffer && pos < max,
00626             "internal_format_payload: precondition violated by caller");
00627
00628   if (length == 0) {
00629     pos = internal_append_str(buf, pos, max, "[PAYLOAD] (empty)\r\n");
00630     return pos;
00631   }
00632
00633   /* payload must be non-NULL when length > 0 */
00634   if (payload == nullptr) {
00635     return pos;
00636   }
00637
00638   uint16_t offset = 0;
00639   while (offset < length) {
00640     /* Start new line with offset */
00641     pos = internal_append_str(buf, pos, max, "[");
00642     pos = internal_append_hex_u16(buf, pos, max, offset);
00643     pos = internal_append_str(buf, pos, max, "] ");
00644
00645     /* Calculate bytes on this line */
00646     uint16_t line_bytes = length - offset;
00647     if (line_bytes > k_bytes_per_line) {
00648       line_bytes = k_bytes_per_line;
00649     }
00650
00651     /* Format hex bytes and ASCII sidebar */
00652     pos = internal_format_hex_line(buf, pos, max, payload, offset, line_bytes);
00653
00654     pos = internal_append_str(buf, pos, max, "\r\n");
00655     offset += line_bytes;
00656   }
00657
00658   return pos;
00659 }
00660
00661 /**
00662  * @brief Format frame header metadata as single text line
00663  *
00664  * @details
00665  * Formats the frame header fields into a human-readable summary line
00666  * including direction indicator, sequence number, payload length, frame
00667  * type, and flags.
00668  *
00669  * @par Output Format
00670  * ```
00671  * [TX] seq=12345 len=64 type=COMMAND flags=ACK|PRI
00672  * [RX] seq=12346 len=128 type=RESPONSE flags=NONE
00673  * ```
00674  *
00675  * @par Field Details
00676  * | Field | Format | Example |
00677  * |-------|--------|---------|
00678  * | Direction | [TX] or [RX] | [TX] |
00679  * | Sequence | seq=N (decimal) | seq=12345 |
00680  * | Length | len=N (decimal) | len=64 |
00681  * | Type | type=NAME | type=COMMAND |
00682  * | Flags | flags=F1\|F2 | flags=ACK\|PRI |
00683  *
00684  *
00685  *
00686  * @pre buf != nullptr
00687  * @pre header != nullptr
```

```
00688  *
00689  * @post Header line written with \\r\\n line ending
00690  * @post pos <= max (invariant maintained)
00691  *
00692  * @ingroup frame_ascii_helpers
00693  * @since Version 1.0.0
00694  */
00695 RX_STATIC_TESTABLE uint32_t internal_format_header(char*                    buf,
00696                                                    uint32_t                 pos,
00697                                                    uint32_t                 max,
00698                                                    const rx_frame_header_t* header,
00699                                                    bool                     is_tx)
00700 {
00701   /* Rule 5: Pre-condition invariant - callers guarantee these conditions */
00702   RX_ASSERT(buf != nullptr && header != nullptr && max != k_empty_buffer && pos < max,
00703             "internal_format_header: precondition violated by caller");
00704
00705   /* Safety guard: prevent null dereference in unit test mode after assert fires */
00706   if (header == nullptr) {
00707     return pos;
00708   }
00709
00710   char flags_buf[k_flags_buf_size];
00711
00712   /* Direction */
00713   if (is_tx) {
00714     pos = internal_append_str(buf, pos, max, "[TX] ");
00715   } else {
00716     pos = internal_append_str(buf, pos, max, "[RX] ");
00717   }
00718
00719   /* Sequence number */
00720   pos = internal_append_str(buf, pos, max, "seq=");
00721   pos = internal_append_decimal_u16(buf, pos, max, header->sequence);
00722
00723   /* Length */
00724   pos = internal_append_str(buf, pos, max, " len=");
00725   pos = internal_append_decimal_u16(buf, pos, max, header->length);
00726
00727   /* Type */
00728   pos = internal_append_str(buf, pos, max, " type=");
00729   pos = internal_append_str(buf, pos, max, rx_frame_ascii_type_name((rx_frame_type_t)header->type));
00730
00731   /* Flags */
00732   pos = internal_append_str(buf, pos, max, " flags=");
00733   (void)rx_frame_ascii_flags_str(header->flags, flags_buf, sizeof(flags_buf));
00734   pos = internal_append_str(buf, pos, max, flags_buf);
00735   pos = internal_append_str(buf, pos, max, "\r\n");
00736
00737   /* Rule 5: Post-condition: pos <= max is guaranteed by helpers' bounds checks. */
00738
00739   return pos;
00740 }
00741
00742 /*
      ============================================================================
00743  * Public Functions
00744  *
      ============================================================================
00745  */
00746
00747 /**
00748  * @brief Convert frame type enum to human-readable string
00749  *
00750  * @details
00751  * Implementation of rx_frame_ascii_type_name(). Uses switch statement
00752  * for O(1) lookup with compile-time string literals.
00753  *
00754  *
00755  *
00756  * @see rx_frame_ascii.h for full documentation
00757  */
00758 const char* rx_frame_ascii_type_name(rx_frame_type_t type)
00759 {
00760   switch (type) {
00761     case k_frame_type_ping:
00762       return "PING";
00763     case k_frame_type_pong:
00764       return "PONG";
00765     case k_frame_type_command:
00766       return "COMMAND";
00767     case k_frame_type_response:
00768       return "RESPONSE";
00769     case k_frame_type_ack:
00770       return "ACK";
00771     case k_frame_type_nack:
00772       return "NACK";
```

```
00773     case k_frame_type_log_message:
00774       return "LOG_MESSAGE";
00775     case k_frame_type_reset_ack:
00776       return "RESET_ACK";
00777     case k_frame_type_reset:
00778       return "RESET";
00779     default:
00780       return "UNKNOWN";
00781   }
00782 }
00783
00784 /**
00785  * @brief Append a single flag name with pipe separator if needed
00786  *
00787  * @details
00788  * Conditionally appends a flag name to the output buffer if the corresponding
00789  * flag bit is set. Manages separator insertion: no separator before first flag,
00790  * pipe separator before subsequent flags.
00791  *
00792  * @par Example Sequence
00793  * 1. flags=ACK|PRI, first call (first=true): appends "ACK", sets first=false
00794  * 2. Same flags, second call: appends "|PRI"
00795  *
00796  *
00797  *
00798  * @pre first != nullptr
00799  *
00800  * @post *first set to false if flag was appended
00801  * @post Separator '|' prepended if not first flag
00802  *
00803  * @ingroup frame_ascii_helpers
00804  * @since Version 1.0.0
00805  */
00806 RX_STATIC_TESTABLE uint32_t internal_append_flag(char*       buffer,
00807                                                  uint32_t    pos,
00808                                                  uint32_t    max,
00809                                                  bool*       first,
00810                                                  uint8_t     flags,
00811                                                  uint8_t     flag_bit,
00812                                                  const char* flag_name)
00813 {
00814   /* Rule 5: Pre-condition invariant - callers always pass valid local bool address */
00815   RX_ASSERT(first != nullptr, "internal_append_flag: first is nullptr - caller must validate");
00816
00817   if (flags & flag_bit) {
00818     if (!*first) {
00819       pos = internal_append_char(buffer, pos, max, '|');
00820     }
00821     pos    = internal_append_str(buffer, pos, max, flag_name);
00822     *first = false;
00823   }
00824   return pos;
00825 }
00826
00827 /**
00828  * @brief Convert frame flags bitmask to human-readable string
00829  *
00830  * @details
00831  * Implementation of rx_frame_ascii_flags_str(). Iterates through known
00832  * flag bits, appending names with pipe separators.
00833  *
00834  *
00835  *
00836  * @note Flags formatted as pipe-separated: "ACK|RETX|PRI"
00837  *
00838  * @see rx_frame_ascii.h for full documentation
00839  */
00840 const char* rx_frame_ascii_flags_str(uint8_t flags, char* buffer, uint32_t buffer_len)
00841 {
00842   if (buffer == nullptr || buffer_len == k_empty_buffer) {
00843     return "";
00844   }
00845
00846   uint32_t pos = 0;
00847
00848   if (flags == k_frame_flag_none) {
00849     pos          = internal_append_str(buffer, pos, buffer_len, "NONE");
00850     buffer[pos] = '\0';
00851     return buffer;
00852   }
00853
00854   bool first = true;
00855
00856   /* Use helper to append each flag */
00857   pos =
00858     internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_requires_ack, "ACK");
00859   pos =
```

```
00860     internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_retransmit, "RETX");
00861   pos = internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_priority, "PRI");
00862   pos =
00863     internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_fec_enabled, "FEC");
00864   pos =
00865     internal_append_flag(buffer, pos, buffer_len, &first, flags, k_frame_flag_soft_nack, "SOFT");
00866
00867   buffer[pos] = '\0';
00868   return buffer;
00869 }
00870
00871 /**
00872  * @brief Format frame as human-readable ASCII string
00873  *
00874  * @details
00875  * Implementation of rx_frame_ascii_format(). Orchestrates header, payload,
00876  * and \\rC formatting using internal helper functions.
00877  *
00878  * @par Implementation Notes
00879  * - Validates all inputs before any output
00880  * - Calls internal_format_header() for metadata line
00881  * - Calls internal_format_payload() for hex dump
00882  * - Appends \\rC footer directly
00883  * - Detects truncation and returns k_rx_err_no_mem
00884  *
00885  *
00886  *
00887  * @see rx_frame_ascii.h for full documentation
00888  */
00889 rx_err_t rx_frame_ascii_format(const rx_frame_t* frame,
00890                                bool             is_tx,
00891                                char*            output,
00892                                uint32_t         output_max_len,
00893                                uint32_t*        output_len)
00894 {
00895   /* Rule 5: Pre-condition validation */
00896   if (frame == nullptr) {
00897     return k_rx_err_invalid_arg;
00898   }
00899   if (output == nullptr) {
00900     return k_rx_err_invalid_arg;
00901   }
00902   if (output_len == nullptr) {
00903     return k_rx_err_invalid_arg;
00904   }
00905   if (output_max_len < k_min_output_size) {
00906     return k_rx_err_no_mem;
00907   }
00908
00909   /* Validate payload length does not exceed frame capacity */
00910   if (frame->header.length > k_frame_max_payload) {
00911     return k_rx_err_invalid_arg;
00912   }
00913
00914   uint32_t pos = 0;
00915
00916   /* Format header metadata */
00917   pos = internal_format_header(output, pos, output_max_len, &frame->header, is_tx);
00918
00919   /* Payload hex dump - use validated length to prevent out-of-bounds reads */
00920   pos = internal_format_payload(output, pos, output_max_len, frame->payload, frame->header.length);
00921
00922   /* CRC */
00923   pos = internal_append_str(output, pos, output_max_len, "[CRC] ");
00924   pos = internal_append_hex_u32(output, pos, output_max_len, frame->crc);
00925   pos = internal_append_str(output, pos, output_max_len, "\r\n");
00926
00927   /* Rule 5: Post-condition: helpers cap pos at output_max_len - 1 via bounds
00928    * checks, so pos < output_max_len is an invariant by construction. */
00929
00930   /* Null terminate */
00931   output[pos] = '\0';
00932
00933   *output_len = pos;
00934
00935   return k_rx_ok;
00936 }
```